Strategies for path-based multicasting in wormhole-routed meshes

Eric Fleury and Pierre Fraigniaud*

Laboratoire de l'Informatique du Parallélisme

Ecole Normale Supérieure de Lyon

F-69364 Lyon Cedex 07, France

August, 1997

Abstract

Multicasting is an information dissemination problem which consists, for a processor of a distributed memory parallel computer, to send a same message to a subset of processors. In this paper, we propose new multicast algorithms for a mesh network using *wormhole* routing with the *path-based* facility. These new algorithms perform generally faster than algorithms previously described in the literature under the same model. We used, as criteria to compare algorithms, the *off-line computation time* necessary to prepare the multicast, and the *communication time* required to complete the multicast.

Key words: multicasting, routing, wormhole, multiprocessor, parallel computers.

* Both authors are supported by the research programs *Architecture des Nouvelles Machines* and *Parallélisme, Réseaux, Systèmes* of the CNRS. Subsets of this paper appeared in the Proceedings of the *23rd Annual International Conference on Parallel Processing*, Chicago, Il, 1994, and in the proceedings of the *6th IEEE Symposium on the Frontiers of Massively Parallel Computing* (Frontier '97), Annapolis, Maryland, 1997.

1 Introduction

Communications among processors of a distributed memory parallel computer are often the main causes of performance degradations. Indeed, in most systems, the "elementary communication time" is still larger than the "elementary computation time". Therefore it is of a main interest to focus on the development of efficient communication strategies for distributed memory multiprocessors. From a single source, three kinds of communication problems can be distinguished:

1. Unicast (one-to-one communication problems),
2. Multicast (one-to-many communication problems), and
3. Broadcast (one-to-all communication problems).

The first problem consists in studying the way of finding routes between processors: it is the routing problem. These routes must satisfy several constraints imposed by the flow control mechanism (store-and-forward, circuit-switched, wormhole, etc.) or the routing function (algorithms as e-cube or XY-routing, prefix routing, interval routing, etc.). For instance, under wormhole routing, a major problem is to insure the absence of deadlock (see [7, 8, 9]), and, for interval routing, a major problem is to limit the size of the routing table while keeping good properties for the routes (see [12]).

The third problem consists in studying algorithms for solving global information dissemination problems as broadcasting, gossiping, or scattering, in which one or all the processors execute a communication pattern involving all the other processors simultaneously. We refer to [13, 14] and [26] for surveys of results in this domain.

In this paper, we study the second class of problems. A *multicast* is an information dissemination problem in which one processor (the source) sends a same message to some other processors (the destinations). If the destination-set is reduced to one processor, the problem turns to be the routing problem, whereas the problem becomes the broadcasting problem if the destination-set consists of all the other processors. The multicast communication problem frequently appears in distributed memory multicomputers in order to insure cache coherence, or to implement data-bases or barrier synchronization mechanisms (see [33]). Of course, many multicasts can occur simultaneously in the network.

The multicast problem has been studied under both store-and-forward routing (see for instance [18, 24]), and wormhole routing (see for instance [2, 10, 19, 21, 22, 23, 27, 30, 32, 34]). In this paper, we focus on the multicast problem under wormhole routing. We will make use of a particular hardware facility called *path-based* [15, 23, 29] that allows intermediate destination to get a copy of the multicasted message. We

will propose new algorithms for the mesh topology. These algorithms will be shown to perform faster than the previously known algorithms developed mainly by Lin, McKinley and Ni in [22], and [23].

The next section describes our communication model and, in particular, the *path-based* facility. In Section 3, we will describe the routing function that we have used on the mesh, and we will explain why we considered such a routing function. Then, we will recall the previously known multicast algorithms on the mesh. In Section 4, we will present a new algorithm which uses a pipeline approach. This algorithm performs faster than the so called dual-path algorithm [22, 23], though with a small increase of the off-line computation time in certain cases. In Section 5, we will derive algorithms which generalize the multi-path algorithm of Lin, McKinley and Ni. These algorithms are based on a different way to "split" the destinations. In Section 6, we will present an algorithm which combines the two previous approaches: pipelining and good splitting. This latter algorithm performs faster than the algorithm based on the pipeline approach, though with a larger off-line computation time. It can be considered as a good trade-off between the time to prepare the data and the time to communicate these data. Section 7 will present the behavior of our algorithms when possible contentions due to other multicasts occur. We will see that our algorithms keep their good behavior in this context. In the same section, we will also study the performances of our algorithm as a function of the ratio propagation time over start-up time. Section 8 is a brief section dealing with huge message size. Finally, Section 9 concludes this paper. We will resume there all the complexities of the several algorithms described in this paper, and we will explain why one of our algorithms, the so-called *Parallel Multi-Column* (PMC) algorithm, seems to be the best protocol for multicasting in meshes.

2 Statement of the problem

Let us consider a multicomputer composed of processors interconnected together by some fixed topology modeled as a *graph* $G = (V, E)$ with *node set* V and *edge set* E. Each node represents a *processing element* (PE), and an edge between two nodes represents the communication link between two neighboring PE's. To each physical communication link (an edge of the graph) is associated one or more *"virtual" channels* in each direction (at least one in each direction). Multiple channels (or virtual channels) are useful to avoid deadlocks [7, 8, 9, 11]. The presence of virtual channels may also allow to increase performances in case of contentions [5].

2.1 Basic hypotheses

In this paper, the discussion is mainly restricted to the 2-*dimensional mesh* topology. This graph is formally defined as follows: a $m \times n$ mesh is a graph $G = (V, E)$ where

$$V = \{(x, y) \in \mathbb{N} \times \mathbb{N} \text{ s.t. } 0 \leq x < n \text{ and } 0 \leq y < m\},$$

and

$$E = \{((x_i, y_i), (x_j, y_j)) \text{ s.t. } |x_i - x_j| + |y_i - y_j| = 1\}.$$

Note that we do not consider the torus network (which is a mesh with "wraparound" links). However a large part of the results of this paper applies to torus networks as well.

The multicast problem can be described formally as follows. A *multicast-set* is a couple (u, K) where $u \in V$ and $K = \{u_1, \ldots, u_k\}, u_i \in V, i = 1, \ldots, k$. The node u is called the *source* node, and the k nodes of K are the *destination* nodes. Given a node u, a *multicast* from u is defined by a multicast-set (u, K): it corresponds to the information dissemination problem in which the node u has to send a same message to all the nodes in K. In the following, we will assume that any node can initiate a multicast to any set of nodes at any time. We will closely follow the basic assumptions stated by Duato in [10], and by Lin and Ni in [23].

Wormhole routing [6, 28]. Message consist of a sequence of *flits*. The header flit(s) of the message governs the route and the remaining flits follow in a pipeline fashion.

Path-based facility [16, 22, 23]. When the flits of a message reach an intermediate destination, they can be copied to the local memory while they continue to flow through the node to reach other destinations. A message is removed from the network when it reaches its last destination.

Multiple headers [23]. Headers consist of a list of destination addresses that must be reached in the specified order. More precisely, a header consists of an ordered sequence of addresses $@(v_1)@(v_2)\ldots@(v_k)$ meaning that the message must go first to v_1, next to v_2, and so on. As soon as a message arrives at an intermediate destination v_i, the address $@(v_i)$ is removed from the header, and the message continues in order to reach the next destination specified by the header, namely $@(v_{i+1})$. To implement the path-based facility at the header level, one can add a 2-bit control field [25] to the data field of each flit. Note that since we will measure the length of the messages in flits, and not in bytes, such an implementation does not affect the calculations presented later in the paper.

All-Port [13]. Each node of the network consist of a processing element (PE) with its local memory, and its router. The all-port model specifies that routers are able to relay multiple messages simultaneously

provided that each incoming message requires a unique outgoing channel leading to a neighboring node. It also assumes that there is an internal channel (a channel connecting the router to the memory of the PE) for each external channel (a channel connecting the router to a neighboring router), and that all the internal channels can be used simultaneously to send and/or receive messages [28].

We have derived efficient multicast algorithms dedicated to the mesh under these assumptions. To compare our algorithms with other algorithms previously derived in the literature under the same model, we have studied two parameters: the off-line computation time to prepare the message (mainly to set the header), and the communication time required by the message to reach its destinations once it enters the network.

2.2 Analysis of the results.

All the previous multicast algorithms derived in the context specified in the previous section made use of the concept of *split-and-sort function*. Given a multicast-set (u, K), a split-and-sort function splits the destination-set into subsets $K_1, K_2, \ldots, K_r$. The multicast algorithm then consist in sending the message to each of these destination-subsets simultaneously, or sequentially. For that purpose, the split-and-sort function determines a header for each subset K_i by sorting the destinations. Each different way of splitting the destination-set, and each different way of sorting the destination-subsets define a different multicast algorithm. The computation time induced by the split-and-sort function will be called the *off-line computation time*. It is a preponderant factor to estimate the multicast algorithm complexity. This time is generally a function of the number of destinations $k = |K|$, and/or of the total number of nodes mn of the mesh (n is the number of columns and m is the number of rows).

To analyze the communication time, we started from a communication model that assumes the time T to transmit a message along a path P to be $T = t_0 + d\delta + L\tau$. (A *path* is an alternating sequence of nodes and edges, beginning and ending with a node. The *length* of a path is the number of edges contained in the path.) In this model, $L = n_{end} + n_{message} + n_{header}$ denotes the size of the message where n_{header}, $n_{message}$, and n_{end} denote the number of flits of the header, the message itself, and the "end of message" signal, respectively. Also, d denotes the length of the path P, t_0 denotes the startup time, that is the time needed by the network interface to setup a communication, δ denotes the time for a flit to be forwarded from a router to a neighboring router (including both switch delay and transfer time), and τ denotes the inverse of the bandwidth of a link, i.e., the elementary propagation time.

Remark. Note that the fact that the total size of the message decreases along its path (because headers

corresponding to intermediate destinations are removed from the list of headers once the message reaches these destinations) does not limit the validity of the model $T = t_0 + d\delta + L\tau$. Indeed, t_0 is required to set up the communication, then $L\tau$ is required for the message to leave the source, and finally $d\delta$ is required for the last flit of the message to be transfered from the source to the destination.

Decomposing δ in $\delta' + \tau$, where δ' is the commutation time, yields a more accurate model $T = t_0 + \delta' d + (L + d - 1)\tau$. Since wormhole-routed networks are usually not distance sensitive (the commutation time is an hardware process which has a negligible effect on the network latency [28]), we have considered $\delta' = 0$. Setting the time needed to transmit a flit to 1, i.e., setting $\tau = 1$, led us to the following communication model:

$$T = \beta + d + L - 1, \tag{1}$$

where β denotes the ratio of the start-up time over the propagation time of a flit from one router to a neighboring router.

Unfortunately, Equation 1 is too complicated to allow simple analyses of multicast algorithms. Indeed, Equation 1 is valid only if there is no congestion. Moreover, most of the multicast algorithms are obtained from heuristics whose behaviors are difficult to describe in a pure analytical way. Therefore, we have mainly used the model of Equation 1 in an experimental environment, and we have performed many tests by simulations. We have performed two different sets of simulations. They are described below. In both cases, we have assumed that $n_{end} = 1$, and each address is encoded by one flit. The message-lengths differ in our several experiments. We have considered 1-flit messages, 10-flit messages, 30-flit messages and 100-flit messages. Comments on longer messages are given in Section 8. In all the analytic examples however, the length of the message to be multicasted is always supposed to be 3 ($n_{message} = 3$).

Terminology. On the figures that report the results of our experiments, the performances of a given algorithm $\mathcal{A}_1$ will be depicted relatively to the performances of another algorithm $\mathcal{A}_2$. This will allow us to precisely compare the efficiency of our algorithms. The *speedup* of $\mathcal{A}_1$ over $\mathcal{A}_2$ is simply the ratio of the experimental communication time of $\mathcal{A}_1$ over the experimental communication time of $\mathcal{A}_2$.

Let us now describe our two sets of experiments.

2.2.1 A simplified analysis

We have first run simulations on a 32×32 mesh. The aim of this first set of experiments is to simulate the behavior of our algorithms in a simple case, that is when only one source is supposed to initiate a multicast. Also, in this first set of simulations, we simplified the communication model of Equation 1. In this simpler

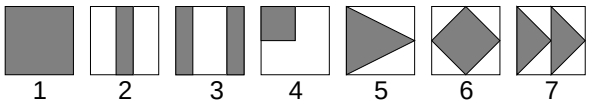


Figure 1: *Destination patterns for our simulations of multicast algorithms. The destinations are randomly uniformly chosen in the grey areas.*

model, β is set to zero, that is the start-up time is supposed to be very small in front of the propagation time. This yields $T = d - 1 + L$. Although this model may not be realistic, it allows to focus on specific types of criteria, mainly the bandwidth required for the communication.

For our first set of simulations, we have fixed the number of destination nodes as 10%, 5% or 1% of the total number of nodes of the mesh. Also, in order to simulate different types of multicasts, we have chosen different patterns of destination-sets as shown in Figure 1 (destinations are chosen in the shadowed area). Pattern 1 corresponds to a uniform distribution of the destinations on the mesh. Patterns 2 to 7 do not necessarily correspond to specific applications. The aim of these patterns is to oversight the behavior of some of our algorithms. Note that some of them just correspond to specific forms of locality (consecutive columns for pattern 2, and blocks for pattern 4 [17]). Of course, the best would be to do experiments based on real applications. However, there is a tremendous number of possible applications, and it is not clear that any of them reflect more than its own behavior. For each pattern, we have chosen uniformly at random the destination nodes inside the subset of nodes defined by the pattern. The source is randomly chosen on the bottom row (we will see later that choosing the source on the bottom row does not affect the performance analysis of the algorithms).

We have also studied three message lengths: 1 flit, 10 flits and 30 flits. Note that ℓ flits does not necessarily mean ℓ bytes. Also, an ℓ-flit message can fill ℓ links between the source and the destination because our experiments assume that one flit is transmitted per simulation step. Even if some applications may multicast very large messages (a matrix row for instance), most of the practical uses of multicast algorithms (synchronization, cache-coherence, etc.) involve small messages. Therefore, we did not run any experiment on very large messages in this first set of simulations, though we will discuss this problem further in the text (see Section 8).

For every percentage of destinations, and for every pattern, we have randomly chosen 100 samples of destinations. For each sample, we have simulated three multicasts, each with a different length of message. We have then computed the mean of the communication times of the simulated algorithms using Equation 1.

We have also computed the global mean, and the standard deviation of the times over all the patterns for every percentage of repartition, and for every message length. We have chosen to report only results concerning 1-flit messages with 10% of destinations, 10-flit messages with 5% of destinations, and 30-flit messages with 1% of destinations. Unless particularly stated, the other results looked similar.

2.2.2 Contention and start-up times

In Section 7, we will present another set of simulations. These experiments assume the model of Equation 1, where several multicasts can occur at the same time, inducing possible contentions between messages. Since these simulations are time consuming, we have considered a 16×16 mesh, with one channel per direction. Each buffer is of size 1 flit. Again, we have considered short messages (1-flit long), medium size messages (10-flit long), and long messages (100-flit long). Note that a message of 10 flits may require a number of channels greater than 1/3 times the diameter of the network, and that a 100-flit message may require a number of channel greater than 3 times the diameter of the network. When two messages content for the use of a channel, one is delayed, and its flits are buffered all along the path corresponding to this message.

We have chosen two different values for β (see Equation 1). In a first scenario, we have considered $\beta = \frac{t_0}{\tau} = 10$. In a second scenario, we have considered $\beta = 100$.

We have also considered two different kinds of encoding for the headers [3, 31]. A first encoding uses variable header size, that is it assumes that one flit encodes a constant number of addresses (actually, in our simulations, we have supposed that 1 flit encodes 4 addresses). The second encoding uses fixed header size, that is a bit-field such that the ith bit is equal to 1 if and only if node i belongs to the destination-set (in our simulations, we have used a bit-field of 8 flits).

3 Previous works

The design and the behavior of multicast algorithms are strongly related to the routing function. On the mesh, the most popular routing function is the XY-routing that consists in routing in the X-direction first and then in the Y-direction. This routing function is deadlock-free for one-to-one communications [7]. However, the path-based hardware facility introduces new dependencies due to the structure of the header which can contain several addresses. For instance, if the first destination of a multicast is reached in the Y-direction, the message might be routed again in the X-direction to reach another destination, creating a $Y \rightarrow X$ dependency, and therefore inducing possible cycles in the channel dependency graph [10]. A first way to solve this problem is to force the split-and-sort function to deal with the destination-set in such a way

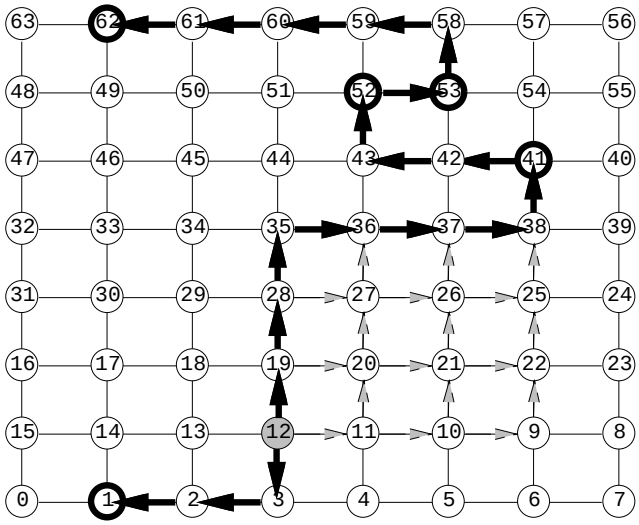


Figure 2: *Labeling of a 8×8 mesh. (Node $(0,0)$ is located on the bottom-left corner.) The source node 12 initiates a multicast to nodes 1, 41, 52, 53, and 62. The arrows indicate the routes followed by the message from its source to its destinations.*

that the new dependencies introduced between the channels do not produce cycles in the channel dependency graph. Another solution is to use a routing function which insures that the split-and-sort function will not produce deadlocks. We adopt the latter solution, according to the works in [22, 23].

3.1 A deterministic routing function on the mesh

Lin and Ni derived in [23] several multicast strategies for hypercubes and meshes. Then Lin, McKinley and Ni focused on the mesh [22]. They did not use XY-routing, but rather defined a new routing function. In order to do this, they assign a label to every node in an increasing order according to an Hamiltonian path. This labeling $\mathcal{L}$ of the nodes of an $m \times n$ mesh is defined by:

$$\text{if } u = (x, y) \text{ then } \mathcal{L}(u) \quad = \quad \begin{cases} y * n + x & \text{if } y \text{ is even} \\ y * n + n - x - 1 & \text{if } y \text{ is odd} \end{cases}$$

where x and y are respectively the column and the row coordinates of the nodes. This labeling constructs an Hamiltonian path whose nodes are labeled consecutively from 0 to $mn - 1$ (see the example of Figure 2).

The path followed by a message is defined by a deterministic routing function which uses the labeling of the Hamiltonian path. We denote this routing function by R_d. It is defined as a function of the node currently holding a message, and the destination node of this message. It returns the neighboring node to

which the message must be forwarded, that is $R_d : V \times V \rightarrow V$. More precisely, if u is the current node, and v is the destination node, then $R_d(u,v) = w$ where

$$\mathcal{L}(w) \quad = \quad \begin{cases} \max\{\mathcal{L}(z) \text{ s.t. } \mathcal{L}(z) \leq \mathcal{L}(v),\ z \text{ neighbor node of } u\} & \text{if } \mathcal{L}(u) < \mathcal{L}(v); \\ \min\{\mathcal{L}(z) \text{ s.t. } \mathcal{L}(z) \geq \mathcal{L}(v),\ z \text{ neighbor node of } u\} & \text{if } \mathcal{L}(u) > \mathcal{L}(v). \end{cases}$$

This routing function insures that, assuming $\mathcal{L}(u) < \mathcal{L}(v)$ (respectively $\mathcal{L}(u) > \mathcal{L}(v)$), a message will be sent to the neighboring node w of u with the highest label (respectively lowest) not higher (respectively lower) than the label of the destination node v. As proved in [23], this routing function is deadlock-free even using the path-based facility. Lin, McKinley, and Ni proposed two multicast algorithms based on this function. These algorithms differ in the way of splitting the destination-set. The sorting is always done according to the labeling of the nodes on the Hamiltonian path.

3.2 Two multicast algorithms

Given a multicast-set (u, K), the two algorithms proposed by Lin, McKinley, and Ni perform as follows:

Dual-path algorithm. *Split* K in two sets K_{inf} and K_{sup} where $K_{inf} = \{v \in K \text{ s.t. } \mathcal{L}(v) < \mathcal{L}(u)\}$ and $K_{sup} = \{v \in K \text{ s.t. } \mathcal{L}(v) > \mathcal{L}(u)\}$. Then *sort* K_{inf} and K_{sup} according to the label of the nodes. Once sorted, $K_{inf} = \{v_1, v_2, \ldots, v_r\}$ with $\mathcal{L}(v_1) < \mathcal{L}(v_2) < \ldots < \mathcal{L}(v_r)$, and $K_{sup} = \{w_1, w_2, \ldots, w_s\}$ with $\mathcal{L}(w_1) < \mathcal{L}(w_2) < \ldots < \mathcal{L}(w_s)$. Then construct two messages (both containing the information to multicast), a first one with a header $@(v_r)@(v_{r-1})\ldots@(v_1)$, and another one with a header $@(w_1)@(w_2)\ldots@(w_s)$. Finally, send these two messages simultaneously from u. The two routes are constructed according to the routing function R_d. Let us consider the example of Figure 2: the node labeled 12 initiates a multicast to the destination-set $K = \{1, 41, 52, 53, 62\}$. The dual-path algorithm splits, and sorts, K in two subsets $K_{inf} = \{1\}$, and $K_{sup} = \{41, 52, 53, 62\}$. If we consider the communication time T to transmit a 3-flit message from node 12 to the nodes of K_{sup}, we find $T = 8 + 16 - 1 = 23$. Indeed $L = 1 + 3 + 4 = 8$, and the length of the path is 16.

The off-line computation time of the dual-path algorithm is $O(k \log k)$ to sort the sets K_{inf} and K_{sup}. In general, the communication time is:

$$\beta + L - 1 + \max(d(u, v_r) + \sum_{i=r-1}^{1} d(v_{i+1}, v_i),\ d(u, w_1) + \sum_{i=1}^{s-1} d(w_i, w_{i+1})).$$

Multi-path algorithm. From the dual-path algorithm, Lin et al. derived another algorithm by splitting K_{inf} and K_{sup} into two subsets, respectively K^1_{inf} and K^2_{inf}, and K^1_{sup} and K^2_{sup}. Subset K^1_{sup} (resp.

K^1_{inf}) contains all the destinations of K_{sup} (resp. K_{inf}) whose x-coordinates are less or equal than the x-coordinate of the source u. Subset K^2_{sup} (resp. K^2_{inf}) contains all the other destinations of K_{sup} (resp. K_{inf}), that is destinations whose x-coordinates are strictly greater than the x-coordinate of the source u. Then the message is sent to the four sets simultaneously through the four output ports of u. Let us consider again the example of Figure 2: the multi-path algorithm splits $K_{sup} = \{41, 52, 53, 62\}$ in $K^1_{sup} = \{62\}$ and $K^2_{sup} = \{41, 52, 53\}$. The time to complete the multicast for the subset K^1_{sup} is $T_1 = 5 + 8 - 1 = 12$ because $L = 1 + 3 + 1$, and the length of the path is 8. The time to complete the multicast for the subset K^2_{sup} is $T_2 = 7 + 11 - 1 = 17$. Hence, the total time is $\max(T_1, T_2) = 17$.

As for the dual-path algorithm, the off-line computation time of the multi-path algorithm is $O(k \log k)$. Its communication time can be the same as the dual-path algorithm if all the destinations belong to the same subset, but it can also be much smaller depending on the repartition of the destinations.

Figure 3 shows simulations of the performances of these two algorithms: we give the speedup of the multi-path algorithm over the dual-path algorithm for the three percentages of destinations and message lengths that we have selected (10% and 1 flit, 5% and 10 flits, 1% and 30 flits). The multi-path algorithm is generally faster than the dual-path algorithm (mean ≈ 1.5 faster). In particular, the multi-path algorithm appears to be very efficient on pattern 3 (almost three times faster than the dual-path algorithm for large percentage of destinations and short message lengths). This is quite natural: for this pattern, the destination-set is generally perfectly split in two destination-subsets which allows to decrease the lengths of the paths used to reach the destinations. On the contrary, on pattern 2, the advantage of the multi-path algorithm is not significant because the two subsets are not well balanced.

Remark. Note that, for both algorithms, the strategies for the sets K_{inf} and K_{sup} are the same. The source u has always two ports reserved for sending messages to higher labels (K_{sup}), and two ports reserved for sending messages to lower labels (K_{inf}). The message is simultaneously sent to the lower and higher labels in both algorithms. The dual-path algorithm is "sequential" in the sense that only one port is used to send to the higher labels, and only one port is used to send to the lower labels. The multi-path algorithm is "parallel" in the sense that all ports are simultaneously used.

In the following, we will only describe algorithms for the higher labels. The strategy for the lower labels will be the same. For this reasons, we assumed that the source leads on the bottom row of the mesh for all our simulations without contention.

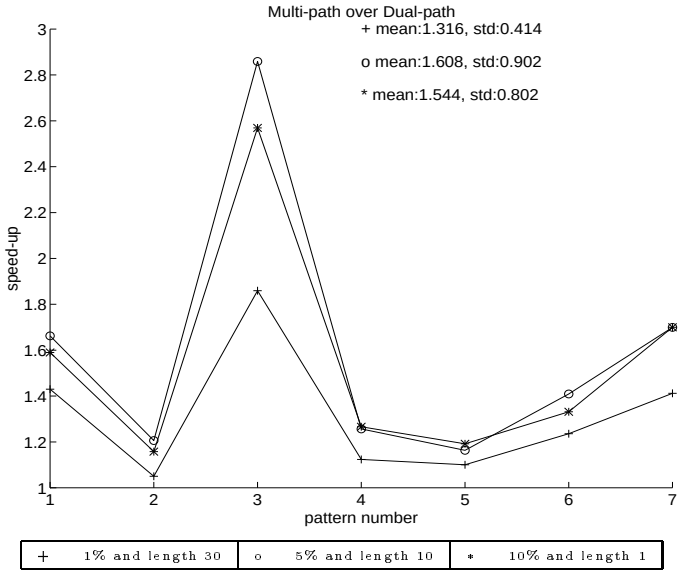


Figure 3: *Speed-up of the multi-path algorithm over the dual-path algorithm.*

3.3 An adaptive routing function for multicasting

Duato noted in [10] that the routing function R_d might be not very efficient since it uses few of all the bandwidth offered by the network. Its main advantage is the deadlock avoidance for multicast, but it is possible to use faster routing functions preserving this property. Duato developed in [10] the theoretical background for the design of deadlock-free *adaptive* routing functions for multicast algorithms. Such a background had to be introduced because of the path-based hardware facility which allows a node to simultaneously read and forward the flits. Roughly speaking, he showed that a routing function may be still deadlock-free even if there are cycles in the channels dependency graph, as soon as there exists a routing sub-function with an extended channel dependency graph free of cycles. This allows him to derive a new deadlock-free adaptive routing function for the mesh.

This function, that we will denote by R_a, is an extension of the routing function R_d defined by Lin et al. (since R_d is deadlock free, R_a is also deadlock free from the results in [10]). For every nodes u and v, let C^H_{uv} (respectively C^L_{uv}) be the set of all the output channels of node u that belong to a shortest path from u to v, and that connect u to a node whose label is not higher (respectively lower) than the label of v. The adaptive routing function R_a is defined by:

$$R_a(u,v) = \begin{cases} C^H_{uv} & \text{if } \mathcal{L}(u) < \mathcal{L}(v) \\ C^L_{uv} & \text{if } \mathcal{L}(u) > \mathcal{L}(v) \end{cases}$$

In other words, if $\mathcal{L}(u) < \mathcal{L}(v)$ (respectively if $\mathcal{L}(u) > \mathcal{L}(v)$), this routing function allows to use any of

the channels belonging to a shortest path between a vertex u to a destination v except when a node with a label higher (respectively lower) than the label of the destination v is reached. For instance, in Figure 2, let us consider all the shortest paths between the node labeled 12, and the node labeled 41. The black arrows represent the deterministic path given by R_d, and the shadowed arrows represent all the additional paths given by the adaptive routing function R_a. (Note that there exists other adaptive deadlock-free routing functions on the mesh: see for instance [21].)

Clearly, the best advantage of an adaptive routing function over a deterministic routing function is the possibility to choose an alternative route in case of high traffic in the network. However, it is pointed out in [1, 4] that this advantage is tedious since adding even few virtual channels can be expensive because it induces large crossbar switches. We do not aim to compare the cost of adaptive versus deterministic routing. Indeed, we are not directly concerned by the use of R_a or R_d because it does not affect the performances of our algorithms in absence of contention (due to other communications different from the studied multicast). All the time analysis presented in this paper in absence of contention make use of the routing function R_a or R_d, but the so-called DPMP algorithm which specifically requires R_a. To simplify experiments in case of multiple traffic inducing contention, we simply used the function R_d in these cases.

Remark. Note that the multi-path algorithm of Lin et al. formally requires that the source is able to choose arbitrarily any of its neighbors as a first intermediate node on the path. Otherwise the two messages destinated to K^1_{sup} and K^2_{sup} (respectively K^1_{inf} and K^2_{inf}) may use a same output link. We adopt the same protocol, that is we assume that we can specify the output channel used by the source to send the message. Once the message is sent, the routing function is applied (R_a or R_d).

The routing function is definitely important. Nevertheless, significant improvements of the multicast algorithms derived by Lin et al. can be obtained by modifying the split-and-sort function. Indeed, using the function R_a instead of R_d will certainly improve performances of these algorithms, but the multicast algorithms themselves are even more important. In this paper we will present three different kinds of multicast algorithms which will be proved to perform faster than the Lin et al.'s algorithms in many cases.

4 The Sequential Multi-Column (SMC) algorithm

Our first algorithm is based on splitting the destination-set in subsets (here groups of columns), and multicasting the message to theses different sets in a pipeline fashion. The term "sequential" refers to the use of only one port at a time among the two ports available to reach higher labels.

To illustrate the main idea of this algorithm, let us consider the example given on Figure 2. The SMC algorithm would perform as follows. We do not send one copy but two copies of the message: a first copy to the nodes of a given subset $K^1_{sup} \subset K_{sup}$, and a second copy to the nodes of another subset $K^2_{sup} \subset K_{sup}$ (for instance $K^1_{sup} = \{41, 52, 53\}$, and $K^2_{sup} = \{62\}$). Now, instead of sending simultaneously K^1_{sup} and K^2_{sup} by two different ports as in the multi-path algorithm, we send first K^1_{sup}, and then K^2_{sup} as soon as the last flit of K^1_{sup} has leaved the source node 12. The time for completing the multicast for the subset K^1_{sup} is $T_1 = 7 + 11 - 1 = 17$ because there are 3 headers, 1 end of message, and $n_{message} = 3$. The last flit of K^1_{sup} leaves the source at time 7, thus the time for completing the multicast for the subset K^2_{sup} is $T_2 = 7 + 5 + 8 - 1 = 19$. Hence, the total time is $\max(T_1, T_2) = 19$. It is less than the time of the dual-path algorithm (which is 23 for the same example). Note that the multi-path algorithm is faster on this example. However, we are here interested in "sequential" algorithms only, and new "parallel" algorithms will be introduced later.

We give below a formal description of the SMC algorithm.

4.1 Split-column function

Recall that we consider the subset K_{sup} only (the case of K_{inf} is similar). Thus, we can suppose, without loss of generality, that the source is on the first row of the mesh with destination-labels greater than the label of the source. For a multicast set $K = (u_0, \{u_1, \ldots, u_k\})$ with $\mathcal{L}(u_0) < \mathcal{L}(u_1) < \ldots < \mathcal{L}(u_k)$, the time of the dual-path algorithm is $T = \beta + L + \sum_{i=0}^{k-1} d(u_i, u_{i+1}) - 1$ where $d(u, v)$ is the distance from u to v. One can do better by considering the split-column-function. An instance of the *split-column-function* for an integer p is a set of p groups of consecutive columns $S_1, S_2, \ldots, S_p$, labeled from 1 to p from left to right, with $S_i \cap S_j = \emptyset$, and such that the cardinality (that is the number of columns) of the different groups differs by at most one.

Now, let us consider the following algorithm: first (at step 1) send the message to destination nodes belonging to S_1; then, when the last flit of the message leaves the source, send (at step 2) the message to destination nodes belonging to S_2; and so on: at step c, send the message to the destination nodes belonging to the group of columns S_c. Step c begins at time $(c-1)(\beta + L)$ because each copy of the message requires $\beta + L$ to get out of the source node. Thus the time to multicast the message to the nodes of $K_c = K \cap S_c$ with this algorithm is:

$$(c-1)(\beta + L) + \beta + L + \sum_{i=0}^{k_c - 1} d(u_i^{(c)}, u_{i+1}^{(c)}) - 1$$

where k_c is the number of destination nodes belonging to the group c ($k_c = |K_c|$), $u_0^c = u_0$ and $K_c =$

$\{u_1^c, u_2^c, \ldots, u_{k_c}^c\}$ in which nodes are ordered according to their label.

This strategy might be not efficient. In particular, if most of the destinations belong to S_p, it would have been more efficient to send the message first to S_p, and then to the other groups. In general, we are looking for a good schedule of the message-sends.

Let $\sigma \in \Gamma_p$ be any permutation of $\{1, 2, \ldots, p\}$, and let σ^{-1} be its inverse. We consider the following multicast algorithm associated to the instance $S_1, S_2, \ldots, S_p$ of the split-column-function, and to the permutation σ: first (at step 1) send the message to destination nodes belonging to $S_{\sigma^{-1}(1)}$. Then, when the last flit of the message leaves the source, send (at step 2) the message to destination nodes belonging to $S_{\sigma^{-1}(2)}$, and so on. At step t, send the message to the destination nodes belonging to the group of columns $S_{\sigma^{-1}(t)}$. In other words, the message to group c is sent at step $\sigma(c)$.

Let us compute the time of such a multicast algorithm. The time

$$D_c(p) \quad = \quad \sum_{i=0}^{k_c-1} d(u_i^{(c)}, u_{i+1}^{(c)}) - 1$$

is the time for the last flit leaving u_0 to reach all the destinations of the group of columns S_c. It does not depend on the permutation. Therefore, for any permutation σ, the time to multicast the message to the nodes of K_c is:

$$T_c(\sigma, p) \quad = \quad \sigma(c)(\beta + L) + D_c(p)$$

because the message is sent to group c at step $\sigma(c)$. For a given permutation σ, the total cost of the multicast is

$$T(\sigma, p) = \max_{1 \leq c \leq p} T_c(\sigma, p)$$

since the multicast is completed when each partial multicast is completed in each group of columns.

Note that the split-column function could be generalized to other kinds of subsets, as group of lines or sub-squares. However, possible contentions may appear in these cases whereas groups of consecutive columns insure the absence of contention during a single multicast.

The performance of the SMC algorithm depends on the number of groups p, and on the schedule determined by the permutation σ for sending the message to these groups. We are therefore interested in finding the solution of the following problem:

Problem 1 *Assume that the number of groups p is fixed, find a permutation $\sigma_0 \in \Gamma_p$ such that: $T(\sigma_0, p) = \min_{\sigma \in \Gamma_p} T(\sigma, p)$.*

This problem is easy to solve because:

Theorem 1 *If a permutation* $\sigma_0 \in \Gamma_p$ *satisfies* $\forall t \in \{1, \ldots, p\}$, $D_{\sigma_0^{-1}(t)}(p) \geq D_{\sigma_0^{-1}(t+1)}(p)$, *then* $T(\sigma_0, p) = \min_{\sigma \in \Gamma_p} T(\sigma, p)$.

Proof. Assume that $\forall t, D_{\sigma_0^{-1}(t)}(p) \geq D_{\sigma_0^{-1}(t+1)}(p)$. Let c_0 such that $T(\sigma_0, p) = T_{c_0}(\sigma_0, p)$. We have $T_{c_0}(\sigma_0, p) = \sigma_0(c_0)(\beta + L) + D_{c_0}(p)$. Two cases can happen with another permutation that modifies the schedule of the group c_0:

1. the message is sent to the group c_0 after the step $t_0 = \sigma_0(c_0)$: this increases $T_{c_0}(\sigma_0, p)$, and thus the total time of the new permutation increases too;

2. the message is sent to the group c_0 before the step $t_0 = \sigma_0(c_0)$: then we will necessarily send the message to a group c such that $D_c(p) \geq D_{c_0}(p)$ after the step t_0, and this will increase the total time of the new permutation. □

Therefore, an optimal permutation for a fixed number of groups can be obtained by sorting the $D_c(p)$'s in a decreasing order. The next point is to find the best possible number of groups p.

Given p, we define $\tau(p)$ as the cost of the fastest multicast algorithm associated to the instance $S_1, S_2, \ldots, S_p$ of the split-column function ($\tau(p) = T(\sigma, p)$ where σ is a best permutation for p groups). To find an optimal p, a solution is to compute all the costs $\tau(p)$ for $p = 1, \ldots, n$ for a $m \times n$ mesh. This solution requires n calculus of the $D_c(p)$, $c = 1, \ldots, p$ which is not reasonable for large n. A classic way to find the minimum of a concave function is to operate by a binary search which requires only $\log n$ steps. Unfortunately the cost function τ is not necessarily concave. However, we will see experimentally that the shape of the curve is regular and nearly concave except in some points. Therefore, when performing a binary search, we will find the minimum with high probability, and, if not, the error $\frac{|\tau(p_{opt}) - \tau(p^*)|}{\tau(p_{opt})}$ will be very small for our approximation p^* of the optimal value p_{opt}.

Assume that destinations are sorted according to their labels (it takes $O(k \log k)$ times). Then, for a fixed value of p, computing $\tau(p)$ takes $O(k)$ for finding the $D_c(p), c = 1, \ldots, p$, plus $O(\min(p \log p, k \log k))$ for sorting the $D_c(p), c = 1, \ldots, p$. Indeed there are at most $\min(p, k)$ non zero values to sort. Since p is smaller than n, computing $\tau(p)$ takes $O(k + \min(n \log n, k \log k))$. There are $\log n$ steps in the binary search, therefore the off-line computation time of the SMC algorithm using the binary search is $O(k \log k + \log n(k + \min(n \log n, k \log k)))$. In particular, if $k = O(n)$, the off-line computation time is $O(k \log k \log n)$, and, if $n = O(k)$, the off-line computation time is $O(k \log k + n(\log n)^2)$.

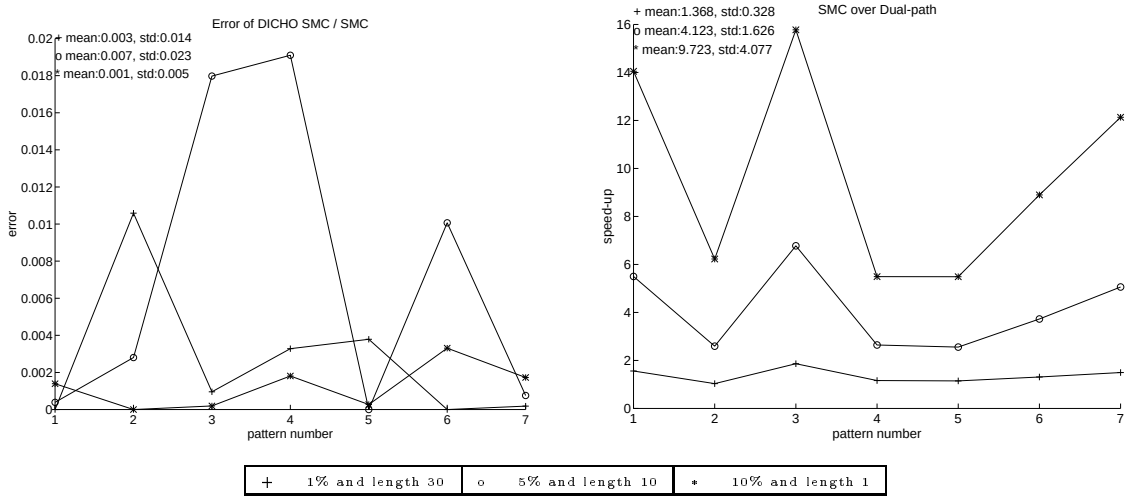


Figure 4: *Error of the binary search in comparison with the exact optimal value (on the left), and speed-up of the SMC-algorithm over the dual-path algorithm (on the right).*

4.2 Simulation

We have performed some simulations to oversight the fact that the binary search gives good results. Once the destination nodes are fixed, we have computed the exact optimal value p_{opt} among all the possible p, and we have computed an approximate value p^* by binary search. The value $\frac{|\tau(p_{opt})-\tau(p^*)|}{\tau(p_{opt})}$ is used as an estimation of our error. These values are reported on the left of Figure 4. This figure shows that the error of the binary search is small, and the exact value p_{opt} is often reached by the binary search (the average value of the error is less than 2% whatever the size of the message is, and whatever the pattern is).

The right hand side of Figure 4 shows that the speed-up of the SMC algorithm (where the number of groups is chosen by the binary search) over the dual-path algorithm is significant. In all cases, the SMC algorithm performs faster. For small messages ($n_{message} = 1$), and a repartition fixed at 10%, it performs, in average, more than 15 times faster for pattern 3, and more than 5 times faster for every pattern. The average speedup value for all the patterns is 9.7 with a standard deviation of 4. If we increase the size of the message ($n_{message} = 30$), the SMC algorithm always performs more than 1.5 times faster than the dual-path algorithm. The shape of the curve remains the same for the different kinds of percentage of repartition, though the gain decreases as the percentage of destinations decreases. For example, for 1%, the global mean is 2.6 for a 1-flit message with a standard deviation of 0.8, and a maximum gain of 6. Anyway, most of the time the SMC algorithm performs much faster than the dual-path algorithm. Note that, as shown on Figure 20, the SMC algorithm performs even faster than the Multi-path algorithm. However, if the performances depend strongly on the off-line computation time, the multi-path algorithm may be preferred.

5 Multi-Path algorithms

The second type of algorithms that we have considered is a generalization of the multi-path algorithm of Lin et al. Our algorithms and the MP algorithm differ in the way of splitting K_{inf} in K^1_{inf} and K^2_{inf}, and K_{sup} in K^1_{sup} and K^2_{sup}. Mainly, the problem is to divide each subset K_{inf} and K_{sup} into two subsets such that the time to send the message in parallel is minimized. We will see that it is possible to improve the split proposed by Lin et al. We are interested in solving the following problem:

Problem 2 *Let $K = \{u_1, \ldots, u_k\}$ be a set of k destinations sorted according to their labels. Consider the multicast set (u_0, K) with $\mathcal{L}(u_0) < \mathcal{L}(u_1)$. Find two routes P_1 and P_2 described by the destination nodes that they should reach, such that:*

1. *No destination node belongs to both P_1 and P_2.*
2. *The two routes cover all the destinations.*
3. *The maximum length of the two routes is minimized.*

For instance, let us consider again Figure 2. The node labeled 12 initiates a multicast to the destination-set $K = \{1, 41, 52, 53, 62\}$. If we split K_{sup} in $P_1 = \{52, 62\}$, and in $P_2 = \{41, 53\}$, we obtain $T_1 = 6+10-1 = 15$ (recall $n_{message} = 3$), and $T_2 = 6 + 9 - 1 = 14$. This gives a total time of 15 which is less than the time of the multi-path algorithm of Lin et al. applied to this example.

5.1 Two heuristic algorithms

We will give below a formal description of two heuristics introduced to find approximations of the solution of problem 2. Indeed, exhaustively checking all possible couples of paths does not yield an efficient algorithm because the number of possibilities is exponential in the number of destinations, leading to a non realistic off-line computation time.

5.1.1 A Dynamic Programming based Multi-Path algorithm (DPMP)

Let us denote by $|P|$ the length of a path P. For any $s \in \{1, \ldots, k\}$, and any $r < s$, we denote by $D(u_r, u_s)$ the minimum of $\max(|P_1|, |P_2|)$ over all the routes P_1 and P_2 satisfying:

1. P_1 and P_2 are originated in u_0, and have other extremities u_r and u_s respectively;
2. for all j, $0 \leq j \leq s$, u_j belongs to exactly one path;

3. all the $u_j, 0 \leq j \leq s$, appear in their path in their order of labeling.

The paths P_1 and P_2 corresponding to the minimum $D(u_r, u_s)$ are denoted by $P_{r,s}$ and $P'_{r,s}$, respectively. We are looking for the value of $D = \min_{0 \leq j < k} D(u_j, u_k)$, and for the corresponding paths which reach this optimal solution.

In the example of Figure 2, we have $D = D(u_3, u_4) = 10$ with the two paths $P_{3,4} = \{u_0 = 12, u_1 = 41, u_3 = 53\}$ and $P'_{3,4} = \{u_0 = 12, u_2 = 52, u_4 = 62\}$. These two paths induce the partition of the destinations into two sets $K^1_{sup} = P_1 = P'_{3,4} = \{u_0 = 12, u_2 = 52, u_4 = 62\}$ and $K^2_{sup} = P_2 = P_{3,4} = \{u_0 = 12, u_1 = 41, u_3 = 53\}$ as given in the example stated just after Problem 2.

There are 2^k possible solutions (each destination can be in either of the two paths). We cannot explore all the possible distributions of the destinations among the two paths, but the following heuristic gives a good approximation of the optimal solution. This heuristic is a greedy approach which states that

1. a good solution for $D(u_j, u_i)$, $1 \leq i \leq k$, $j < i - 1$, is obtained from good solutions for the $i - 2$ problems $D(u_j, u_{i-1})$, $j = 1, \ldots, i - 2$, by adding a path from u_{i-1} to u_i, and

2. a good solutions for $D(u_{i-1}, u_i)$, $1 \leq i \leq k$, is obtained from good solutions for the $i - 2$ problems $D(u_j, u_{i-1})$, $j = 1, \ldots, i - 2$, by adding a path from u_j to u_i.

More precisely, we set $D^*(u_0, u_1) = d(u_0, u_1)$, $P_{0,1} = \emptyset$, and $P'_{0,1}$ = the path from u_0 to u_1 generated by the routing function. Then, we compute, for $i > 1$,

$$\begin{cases} \forall j < i - 1,\ D^*(u_j, u_i) = \max(|P_{j,i-1}|, |P'_{j,i-1}| + d(u_{i-1}, u_i)) & \text{(i)} \\ D^*(u_{i-1}, u_i) = \min_{0 \leq j < i-1} \max(|P_{j,i-1}| + d(u_j, u_i), |P'_{j,i-1}|) & \text{(ii)} \end{cases}$$

This heuristic takes into account only a few ways to obtain two paths, one ending in i, and the other in j. Equation (i) considers the case $j < i - 1$. In this case, we construct the two paths from the two previous paths ending in j and $i - 1$ by adding a shortest path from $i - 1$ to i. Equation (ii) considers the case $j = i - 1$. In this case, we construct the two paths from the best solution among the $i - 1$ pairs of paths $(P_{j,i-1}, P'_{j,i-1})$ by adding a shortest path from j to i.

In time $O(i)$, all the values $D^*(u_j, u_i)$, $0 \leq j < i$, can be computed assuming that we know all the $D^*(u_j, u_{i-1})$ for $0 \leq j \leq i - 1$. Therefore, we can compute an approximated solution of problem 2 in time $O(k^2)$. This will be the off-line computation time of the DPMP algorithm.

Example. Let us consider the example of Figure 2 again. First, we set $D^*(u_0, u_1) = d(12, 41) = 7$, $P_{0,1} = \emptyset$, and $P'_{0,1} = \{u_0 = 12, u_1 = 41\}$. Then, we get two possible solutions: $D^*(u_0, u_2) = 10$ and

$D^*(u_1, u_2) = 7$. After the third phase, we get $D^*(u_0, u_3) = 11$, $D^*(u_1, u_3) = 7$, and $D^*(u_2, u_3) = 9$. Let us detail the fourth phase: from Equation (i), our heuristic generates $D^*(u_0, u_4) = 16$, $D^*(u_1, u_4) = 12$, and $D^*(u_2, u_4) = 12$; from Equation (ii), our heuristic chooses $D^*(u_3, u_4)$ as the minimum of three possible solutions corresponding to $D^*(u_0, u_3)$, $D^*(u_1, u_3)$, and $D^*(u_2, u_3)$, plus the additional term corresponding to the length of the additional path to reach u_4 from u_0, u_1, and u_2, respectively. Note that this pruning reduces the exponential search to linear. We get $D^*(u_3, u_4) = 10$ generated from the solution $D^*(u_2, u_3)$. In this case, our heuristic finds the optimal solution though it might have not been the case for other situations.

Remark. As we will see later, this heuristic is quite efficient in the sense that it gives good approximation of the optimal solution of Problem 2. Unfortunately, it may happen that the two routes corresponding to this solution overlap, and create contentions. The following algorithm solves this problem.

5.1.2 The Window based Multi-Path algorithm (WMP)

To avoid possible contentions that may occur when using the DPMP algorithm, we will consider another way to approximate the best solution of Problem 2. Roughly speaking, the multi-path algorithm of Lin et al. groups the destinations of K_{sup} in two regions which correspond to $A \cup A'$ and $B \cup B'$ on Figure 5. In most cases, these two regions are not balanced in term of path lengths. The WMP algorithm consists in splitting the destination nodes in another way. We group the destinations into two regions that are represented in white and grey on Figure 5. These two regions are formally defined by:

- an integer $\delta = \pm 1$;
- a node v that belongs to the same column as the source u_0, and such that v has a label and a row coordinate greater than the ones of u_0.

On Figure 5, if $\delta = +1$, we get two regions: $R_1 = A$ and $R_2 = A' \cup B' \cup B$. If $\delta = -1$, we get two regions: $R_1 = A \cup A' \cup B'$ and $R_2 = B$. There are $2m$ possible ways of splitting the mesh, one for each pair (v, δ).

The two regions R_1 and R_2 defined by a couple (v, δ) allow to construct two paths P_1 and P_2 that go through all the vertices of R_1 and R_2, respectively, and without contention between them. Of course, given R_1 and R_2, the paths are built so that they are as short as possible with respect to the label ordering. We choose, as an approximation of the solution of problem 2, the couple (R_1, R_2) which gives P_1 and P_2 such that $\max(|P_1|, |P_2|)$ is as small as possible. This choice is made by looking at the $2m$ possible ways of splitting the mesh: 2 choices for δ, and m choices for node v.

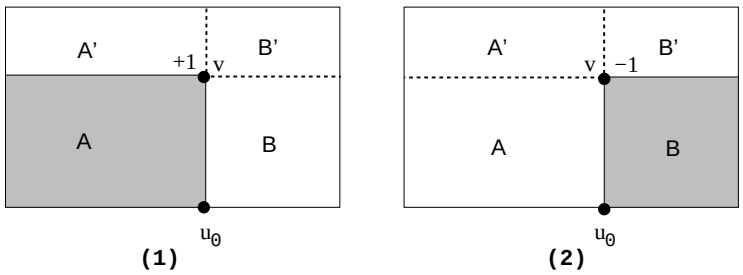


Figure 5: *A couple (v, δ) defines two regions R_1 and R_2 ($R_1 = A$, and $R_2 = A' \cup B' \cup B$ if $\delta = +1$, whereas $R_1 = A \cup A' \cup B'$, and $R_2 = B$ if $\delta = -1$). There are $2m$ possible ways of splitting the mesh, one for each pair (v, δ).*

The off-line computation time of the WMP algorithm is $O(k(\log k + m))$ because sorting needs $O(k \log k)$ and there are $O(m)$ possible ways of splitting the mesh in two regions as defined above.

In the next section, we produce experiments to compare the DPMP algorithm (contentions may happen) with the WMP algorithm (which insures absence of contention).

5.2 Simulations

We have performed simulations to compare our two heuristic algorithms. The conditions are the same as in the previous sections. If we do not consider contentions, the DPMP algorithm performs faster that the WMP algorithm (see Figure 6). More precisely for a 10% repartition, the average gain is 1.2 for 1-flit messages, and the standard deviation is roughly equal to 0.2. The DPMP algorithm could run two times faster for pattern 3. For the other percentages of repartition, the shapes of the curves are the same. In few cases, however, we noticed that the WMP algorithm performs faster than the DPMP algorithm, but never more than 1.5 faster.

We have also compared the multi-path algorithm with our two heuristic algorithms. We noticed that the WMP algorithm always performs faster than the multi-path algorithm. For instance, on Figure 7, for a 10% repartition, the maximum gain is 2, and the average of the speed-up is 1.5 with a standard deviation of 0.3.

Finally, Figure 8 compares the multi-path algorithm with the DPMP algorithm. As opposed to the WMP algorithm, the DPMP algorithm does not always perform faster than the multi-path algorithm (in some cases – less than 2% – the multi-path algorithm performs up to 1.25 time faster than the DPMP algorithm). However the global mean shows that the DPMP algorithm runs roughly twice faster than the multi-path algorithm in average.

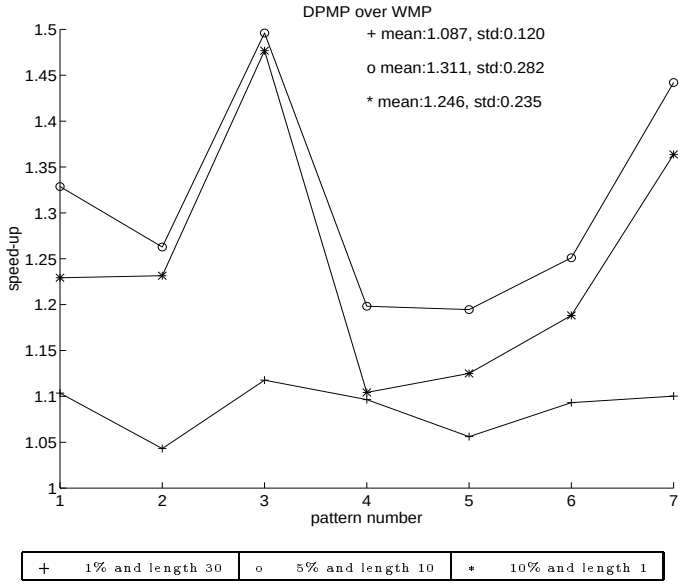


Figure 6: *Speed up of the DPMP algorithm over the WMP algorithm.*

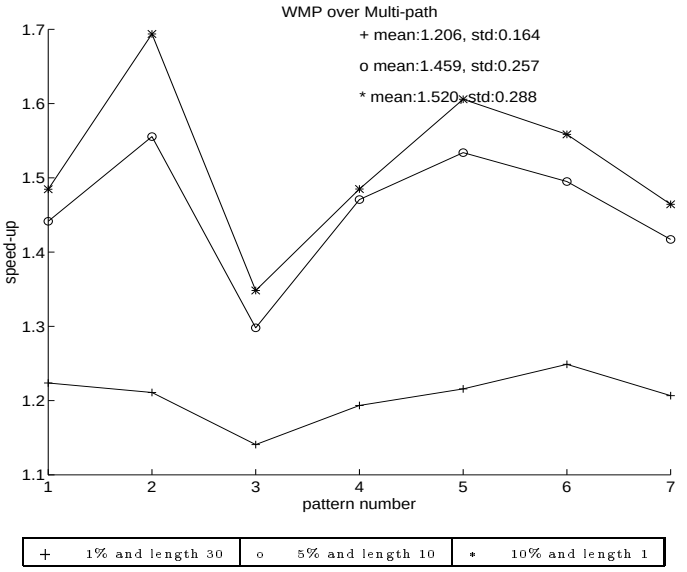


Figure 7: *Speed up of the WMP algorithm over the multi-path algorithm.*

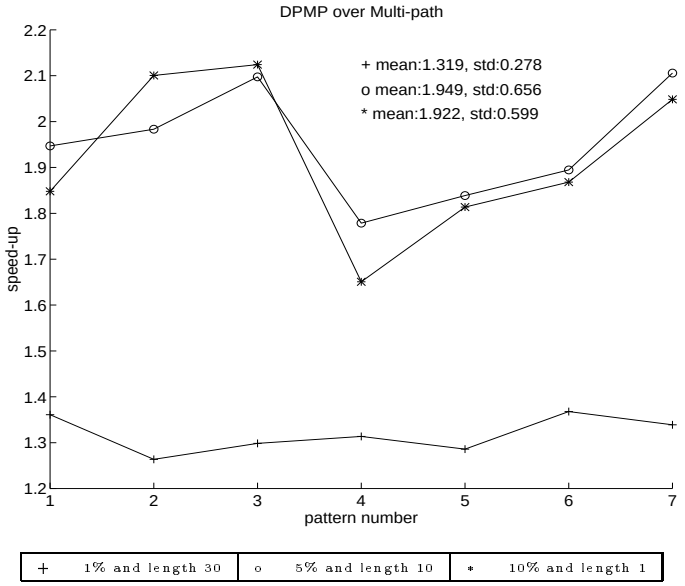


Figure 8: *Speed up of the DPMP algorithm over the multi-path algorithm.*

6 Parallel multicast algorithms

In this section, we will show how it is possible to combine the two first approaches described in the previous sections that are:

1. splitting the mesh in groups of columns; and
2. using a multi-path algorithm.

We will consider two different combinations which give respectively the PMC algorithm (for Parallel Multi-Column), and the MPMC algorithm (for Multi-Path Multi-Column). The first algorithm is based on the sequential multicolumn (SMC) algorithm, while the other is based on both the SMC algorithm and the window based algorithm (WMP).

6.1 The Parallel Multi-Columns (PMC) algorithm

The simple idea of this algorithm is the following: first find an appropriate number p of groups of columns as for the SMC algorithm (this number can be found via a the binary search), and then split the mesh in p groups of consecutive columns (see Figure 9(1)). Then, again as in the SMC algorithm, sort the groups by decreasing order of the values of $D_c(p)$. Now, instead of performing p steps as the SMC algorithm, the goal of the PMC algorithm is to perform in $p/2$ steps. Ideally, if $D_c(p)$ and $D_{c'}(p)$ are the two larger costs, the PMC algorithm would simultaneously send the message by two ports to the groups c and c'. Then, in a

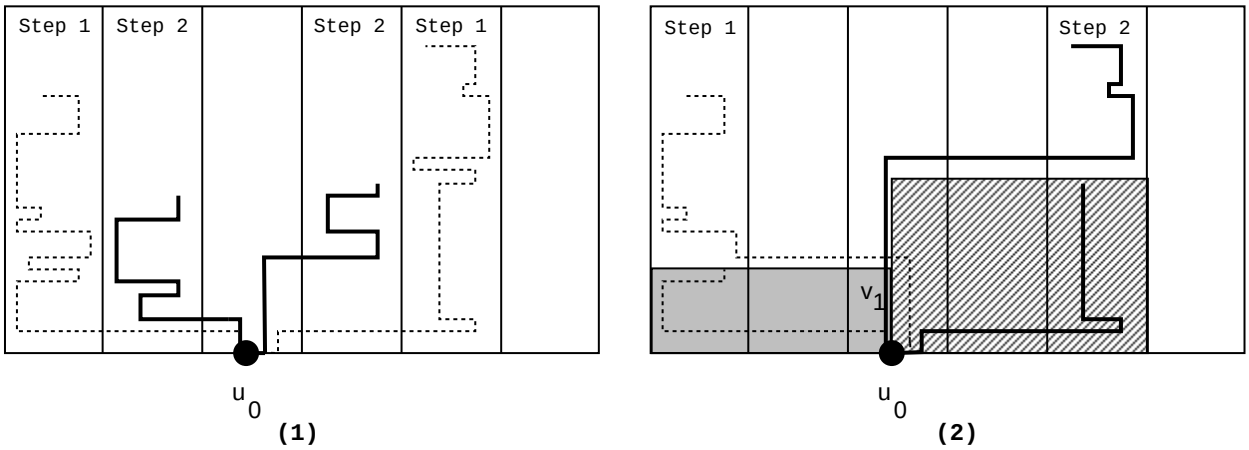


Figure 9: *The use of the PMC algorithm (on the left), and of the MPMC algorithms (on the right). Node u_0 is the source of the multicast.*

second step, if $D_{c''}(p)$ and $D_{c'''}(p)$ are the two next larger costs, the PMC algorithm would simultaneously send the message by two ports to the groups c'' and c''', and so on. For instance, on Figure 9(1), we have assumed that the first and fifth columns are the columns with the larger costs. Therefore, the first step of the PMC algorithm consists in sending the message to the destinations lying in these two columns. We have assumed that the second and fourth columns are the columns with the larger costs among all columns but the first and the fifth. Therefore, the second step of the PMC algorithm consists in sending the message to the destinations lying in these two columns as soon as possible, that is as soon as the output ports of the source are freed.

Unfortunately, it may happen that c and c' (or c'' and c''') are such that the same port has to be used to reach the first destination of each group. Therefore, we use the following strategy. At the first step, let $D_c(p)$ be the maximum cost, and, then, let $D_{c'}(p)$ be the next larger cost such that the message to groups c and c' can be sent through two different ports. Remove $D_c(p)$ and $D_{c'}(p)$ from the list of costs, and repeat the same operation. If, at some moment, the choice of c' is impossible, then the PMC algorithm turns to be the SMC algorithm.

The off-line computation time of this algorithm is the same as the off-line computation time of the SMC algorithm, that is $O(k \log k + \log n(k + \min(k \log k, n \log n)))$.

6.2 Multi-Path Multi-Column (MPMC) algorithm

Again, as for the SMC and the PMC algorithms, we split the mesh in groups of columns (see Figure 9(2)). However, instead of computing the appropriate number of column groups as in the SMC and PMC algorithms, this number of groups is computed by considering the cost of a multi-path algorithm *inside* each group. The MPMC algorithm can be seen as the dual of PMC algorithm. More precisely, it performs as follows.

Once the number p of groups has been fixed, $D_c(p)$ is set as the cost of a multi-path algorithm applied to the group c (recall that the PMC algorithm used $D_c(p)$ as a cost of a single path algorithm applied to group c). Sort again the $D_c(p)$ in a decreasing order. Then proceed step by step in choosing the groups in this order, as in the SMC algorithm. However, each step consists now of an application of a multi-path algorithm inside the considered group. There is no contention as soon as there is no contention inside each group. Actually, we have chosen the WMP algorithm inside each group for that purpose. For instance, assume that the first column from the left on Figure 9(2) is the column with the largest cost. The WMP algorithm is applied to the set of destinations lying in this column. Then, if the fifth column from the left is the column with second largest cost, the WMP algorithm is applied to the set of destinations lying in this column. And so on.

The off-line computation time of this algorithm is the same as the off-line computation time of the PMC algorithm excepted that computing $D_c(p)$ for a fixed p takes $O(mk)$ instead of $O(k)$ since we use the WMP algorithm inside each group. That gives an off-line computation time of $O(k \log k + \log n(mk + \min(k \log k, n \log n)))$.

6.3 Simulation

We have performed simulations to compare the MPMC algorithm with the PMC algorithm (see Figure 10). PMC performs faster in 75% of the cases but the gain is small in general (indeed the total mean overall the simulations is 1 with a standard deviation lower than 0.2).

We have also compared the PMC algorithm with the multi-path algorithm (see Figure 11). The speed up of the PMC algorithm over the multi-path algorithm can be quite large (almost 10). The PMC algorithm is almost always faster than the multi-path algorithm.

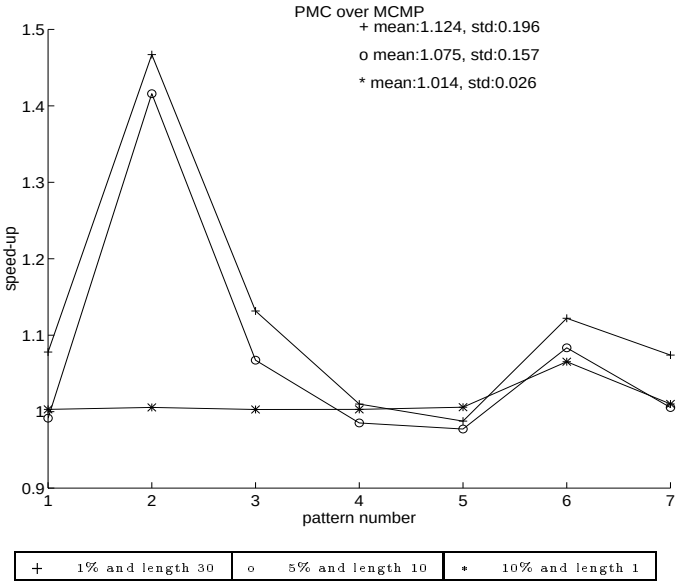


Figure 10: *Speed-up of the PMC algorithm over the MPMC algorithm.*

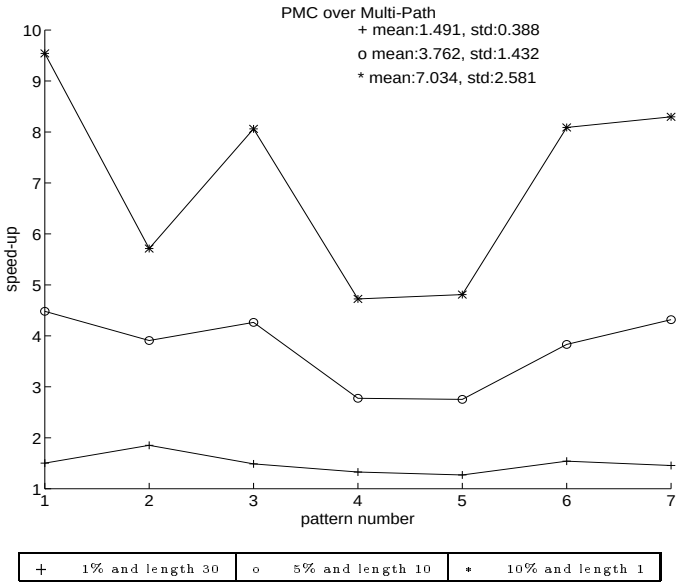


Figure 11: *Speed-up of the PMC algorithm over the multi-path algorithm.*

7 Behavior in case of contentions and large startup times

In the previous sections, we have considered the case of a single multicast. However, it commonly happens that many nodes initiate several multicasts simultaneously. In this case many contentions may occur, and the behavior of our algorithms can be modified. In this section, we will present experimental results dedicated to this problem. Similarly, all experiments presented before have been done assuming that the startup time is very small in front of the propagation time. Since this does not hold in all systems, we have considered two sets of experiments in which the startup time dominates the propagation time by a factor of, respectively, 10 and 100.

We have compared the DP, MP, WMP, PMC and MPMC algorithms. Among all our algorithms, we have selected the PMC algorithm, and the MPMC algorithm because they offer the best performances when there is no contention in the network due to other multicasts. We have also selected the WMP algorithm in order to compare its behavior with the MP algorithm.

7.1 Additional traffic

Figure 12 presents the average additional traffic generated by the algorithms. This additional traffic is computed as in [20], that is by subtracting the number of destinations from the number of channels involved in the multicast. Both figures correspond to variable header size. For fixed header, the shapes of the curves are similar. Our results corroborate the results in [20] concerning both DP and MP algorithms. The average additional traffic does not depend on the length of the messages but for the PMC and MPMC algorithms (because the number of columns involved in this algorithm is a function of the message length, and of the ratio startup time over propagation time). For these two algorithms, we have depicted their average additional traffic for small message size (1-flit) because this size is the one that produces the most average additional traffic.

Let us analyze Figure 12(a), that is the one with a small ratio startup time over propagation time ($\beta = 10$). The MPMC algorithm generates more additional traffic than any other algorithm whatever the number of destination is. Minimizing the maximum of the path lengths by using the WMP algorithm inside each group of columns does not minimize the additional traffic. This is natural, and pipelining actually even increases this phenomenon. For a reasonably small number of destinations (less than 110), the PMC algorithm is the algorithm which generates the less additional traffic. However, this additional traffic does not decrease as well as for the other algorithms when the number of destinations increases. This is due to the fact that the PMC algorithm (and the MPMC algorithm too) continuously uses pipelining, and therefore

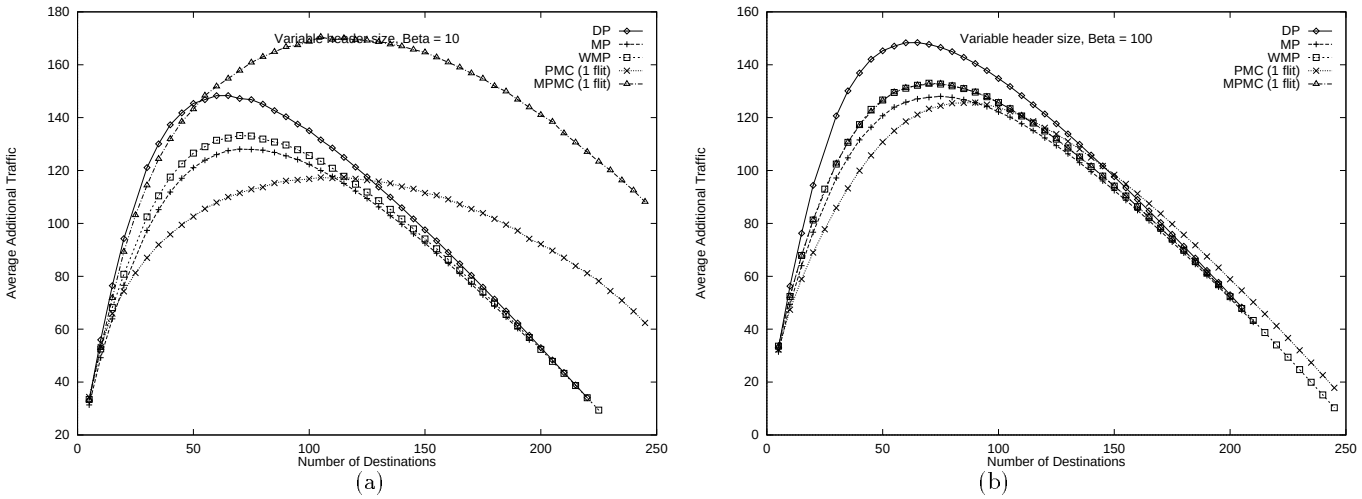


Figure 12: *Average additional traffic as a function of the number of destinations for the five algorithms DP, MP, WMP, PMC, and MPMC. One the left hand side, the ratio startup time over propagation time is set to 10, whereas it is set to 100 on the right hand side.*

generates extra additional traffic. The WMP algorithm has an additional traffic that is a bit larger than the PMC algorithm.

Figure 12(b) reports the use of a large ratio β. For this ratio, the behavior of all the algorithms becomes similar. DP have the most important additional traffic. We will see later however that a small additional traffic does not necessarily imply an efficient behavior in case of contention, and that algorithms generating an important additional traffic may behave well in practice. Note that the PMC algorithm generates less traffic than any other algorithm when the number of destinations is relatively small (less than 85).

7.2 Latency as a function of the injection rate

As in [20], we have also performed experiments on the average network latency as a function of the injection rate. The average network latency is the mean of the transmission times of the messages from their sources to their destinations. Every node generates messages with an average time between two send-instructions that is equals to the specified injection rate. In Figures 13, 14 and 15, we have depicted the average network latency of the five multicast algorithms considered in this section as a function of the injection rate (when the behavior of the MPMC algorithm was similar to the behavior of the PMC algorithm, we have chosen to not report the result of the former in order to clarify the figures). The number of destinations was fixed to 10.

In Figure 13, the ratio β is set to 10. One can check on Figure 13(a) that the behaviors of all the

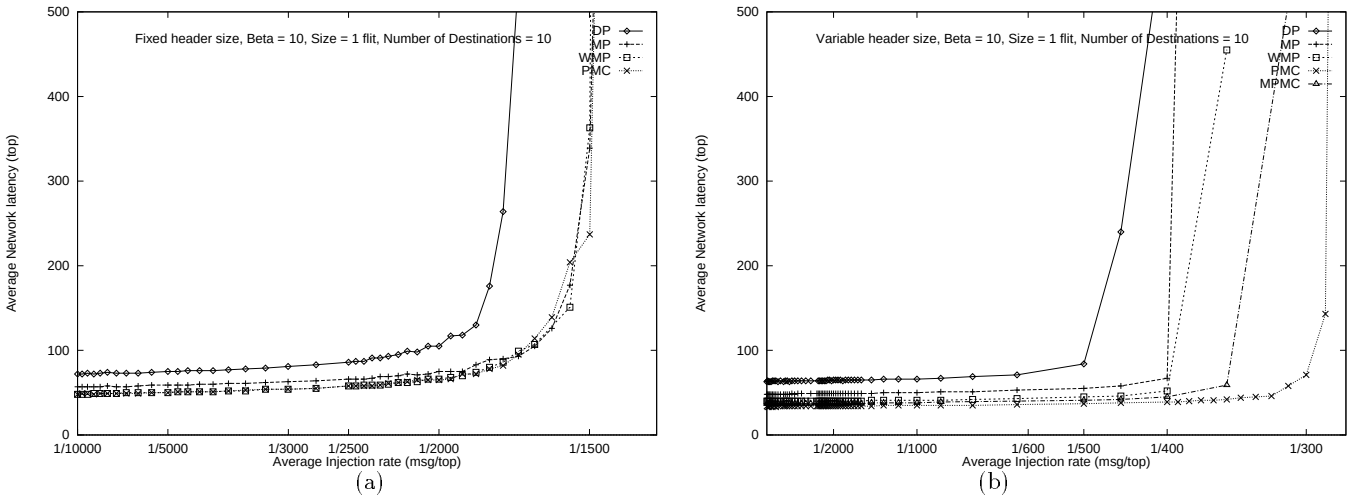


Figure 13: *Average network latency as a function of the injection rate. The ratio $\beta = 10$, the number of destinations = 10, and the size of the messages = 1 flit. On the left hand side, the header size is fixed, whereas it is variable on the right hand side.*

algorithms, but the DP algorithm, are very similar when the header size is fixed. However, both PMC and WMP algorithms obtain better performance when the injection rate is smaller than 1/1750. They are both about 11% to 15% faster than the MP algorithm. For larger injection rate, the network becomes saturated, and the WMP algorithm obtains better performances though very close to the performances of the MP algorithm. The DP algorithm saturates first in any case. For variable header size, one can check on Figure 13(b) that the PMC algorithm obtains better performances. It saturates lately (injection rate about 1/300) and it is 30% faster than the MP algorithm which saturates for an injection rate of about 1/400. Both WMP and MPMC algorithms obtain good performances. They are roughly 17% faster than the MP algorithm. Again, the DP algorithm saturates first.

For medium message size (10 flits), the shapes of the curves are similar to the shapes of Figure 13, and the same analyze as the one for 1-flit messages holds (see for example Figure 15(a)). For long messages (100 flits), and for a startup $\beta = 10$, the performances of the five algorithms are depicted on Figure 14. When the header size is fixed (Figure 14(a)), the performance of the MP, WMP, and PMC algorithms look similar for an injection rate smaller than 1/23500. For a larger injection rate, both WMP and MP algorithms obtain comparable results, and, once the networks becomes saturated, results becomes chaotic. For variable header size, one can check on Figure 14(b) that the performances of the MP, WMP, and MPMC algorithms are again similar, though the behavior of the algorithms are less regular in this context than for fixed header size. The PMC algorithm obtains low performances that are comparable to the performances of the DP algorithm. However, the PMC algorithm saturates later than the DP algorithm.

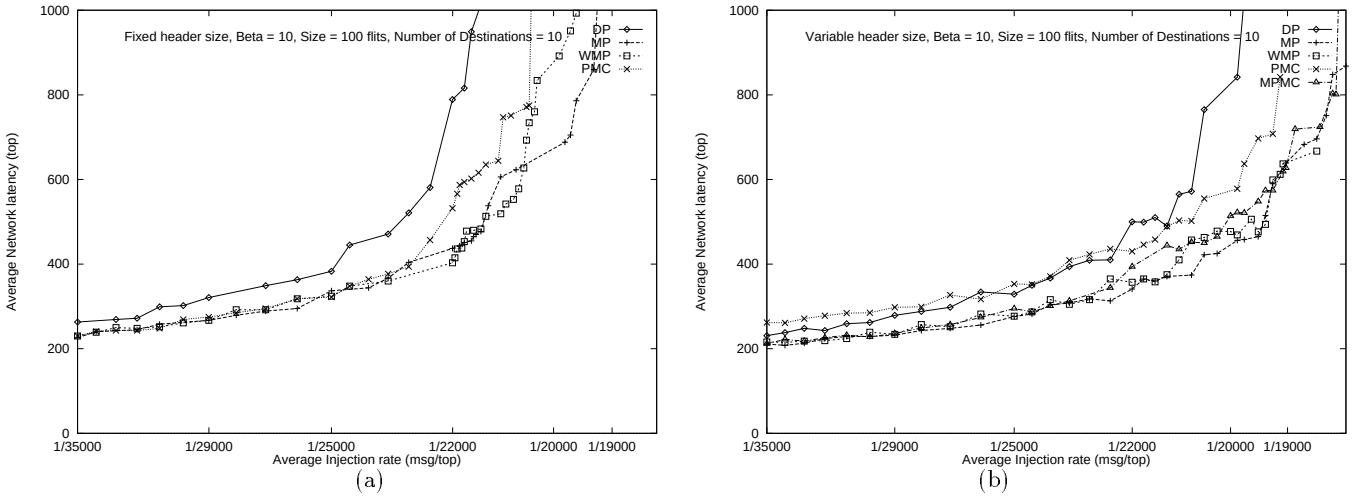


Figure 14: *Average network latency as a function of the injection rate. The ratio $\beta = 10$, the number of destinations = 10, and the size of the messages = 100 flit. On the left hand side, the header size is fixed, whereas it is variable on the right hand side.*

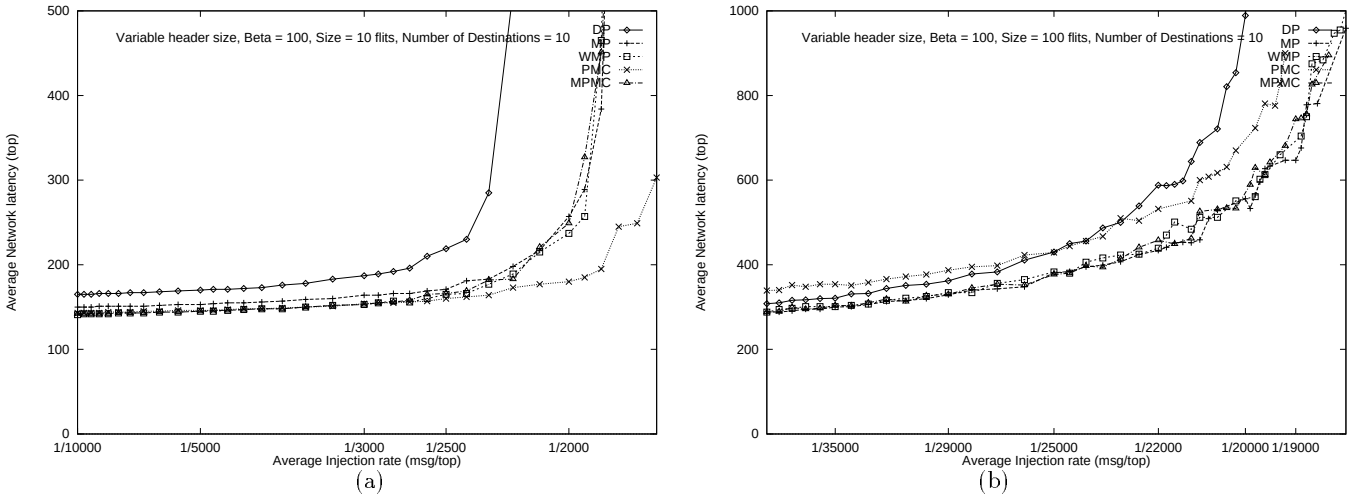


Figure 15: *Average network latency as a function of the injection rate. The ratio $\beta = 100$, the number of destinations = 10, and the header size is variable. On the left hand side, messages are 10-flit long whereas, on the right hand side, they are 100-flit long.*

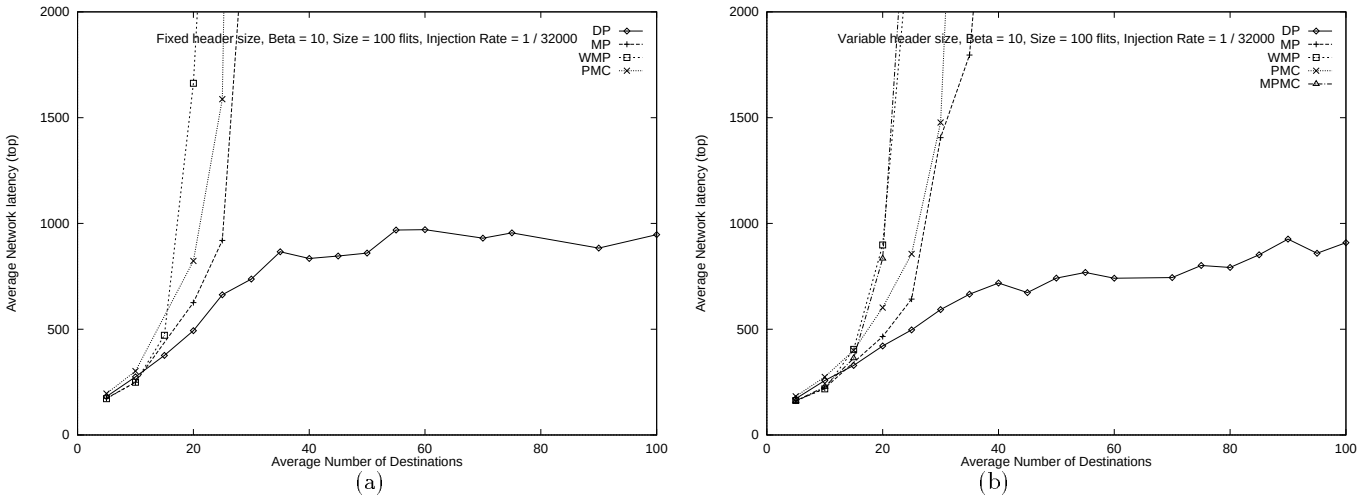


Figure 16: *Network latency as a function of the number of destinations. The message size is 100 flits, the ratio $\beta = 10$, and the injection rate = 1/32000. On the left hand side, the header has a fixed size whereas, on the right hand side, the header has a variable size.*

Figure 15 depicts performances of the five algorithms when the ratio β is large ($\beta = 100$). We have used variable header size. On Figure 15(a), we have presented the performances for short message (size = 10 flits). The WMP, PMC, and MPMC algorithm perform better than the MP algorithm, and the PMC algorithm saturates after the other algorithms, for an injection rate equals to 1/2000. The DP algorithm saturates first. On Figure 15(b), when the message size is equal to 100 flits, the MP, WMP and MPMC algorithms obtain similar performance. The PMC algorithm performs better than the DP algorithm when the injection rate is less than 1/25000, and thus it saturates later. When the header size is fixed, the curves have the same shapes than the curves discussed before, and thus we have decided to not present them.

7.3 Latency as a function of the number of destinations

Figures 16, 17 and 18 present experimental results on the average network latency as a function of the number of destinations.

On Figure 16(a), the injection rate is fixed to 1/32000 messages per time unit, the ratio β is set to 100, and the message size is equal to 100 flits. The algorithms MP, WMP, and PMC have a similar behavior. Among them, the MP algorithm obtains better results when the number of destinations is greater than 15. However, the three algorithms saturate very fast for a small number of destinations (less than 25). For a larger number of destinations, and for long messages, Figure 16(a) shows that the DP algorithm is definitively the best (certainly because it creates less contention). Figure 16(b) presents a similar experiment but with

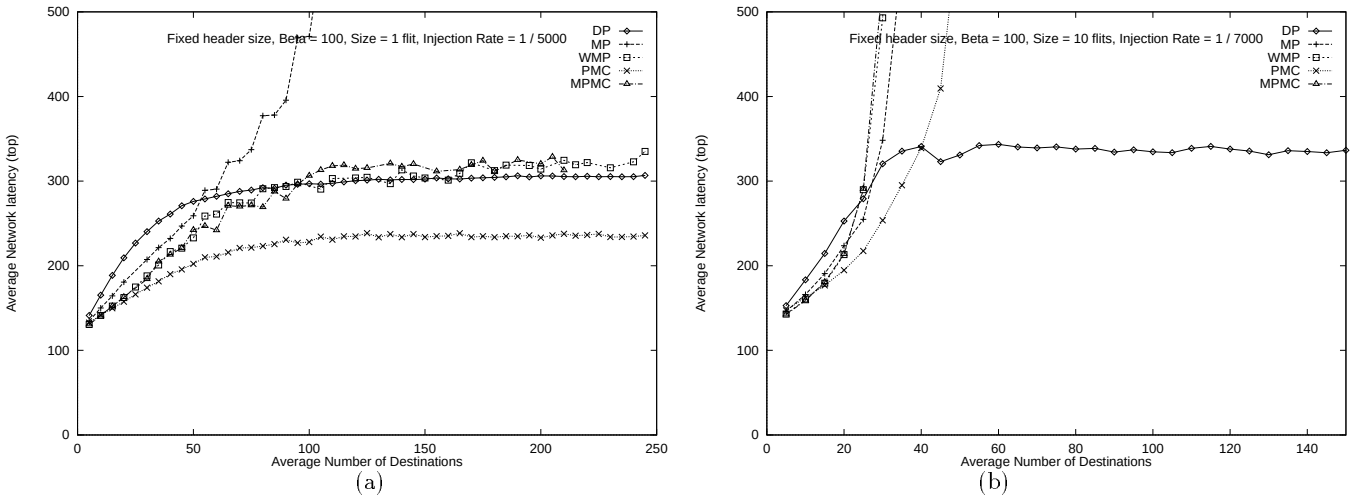


Figure 17: *Network latency as a function of the number of destinations. The ratio $\beta = 100$, and we have used fixed header size. On the left hand side, the message size is set to 1 flit whereas it is set to 10 flits on the right hand side.*

fixed header size. The same analyze as before holds in this case.

Figure 17 presents experimental results for large startup ($\beta = 100$), and for fixed header size. For small message size (1 flit), one can check on Figure 17(a) that the PMC algorithm obtains the best performance. Actually, it performs even better than the DP algorithm. In fact, the DP, WMP, and MPMC algorithms obtain comparable performances. The MP algorithm saturates faster when the number of destinations is greater than 50. On Figure 17(b), we have considered medium size messages (10 flits). The PMC algorithm obtains the best performance when the number of destinations is less than 40. Then the DP algorithm turns to be more efficient for a large number of destinations.

Figure 18 presents experimental results on the average network latency as a function of the number of destinations when messages have a variable header size. Figure 18(a) is dedicated to small messages, and small startup times (1 flit, and $\beta = 10$). Both PMC and MPMC algorithms outperform all other algorithms. The other algorithms saturate faster: the MP algorithm saturates for about 150 destinations, and the DP and WMP algorithms saturate for about 180 destinations. Figure 18(b) deals with medium size messages, and with large startup (10 flits, and $\beta = 100$). The PMC algorithm obtains again the best performances, but only when the number of destinations is less than 140. For a larger number of destinations, both DP and PMC algorithms begin to saturate. The other algorithms have similar behavior, although the MP algorithm saturates first.

We will resume all these experiments in the concluding section.

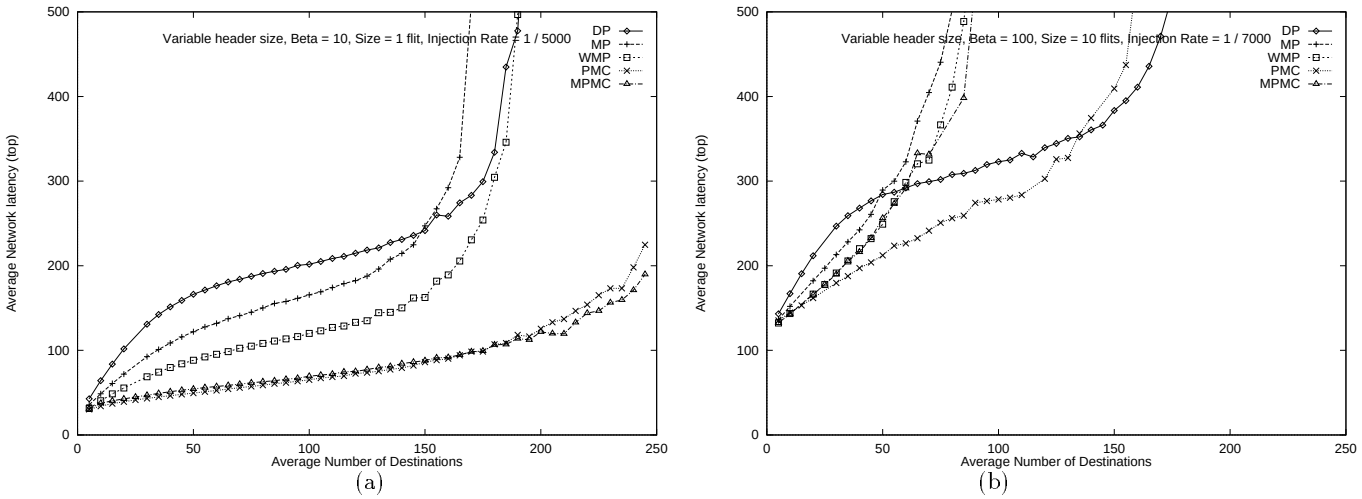


Figure 18: *Network latency as a function of the number of destinations. We have used variable header size. On the left hand side, the ratio β has been fixed to 10, the injection rate was fixed to* 1/5000*, and we have considered 1-flit messages. On the right hand side, the ratio β was fixed to 100, the injection rate was fixed to* 1/7000*, and we have considered 10-flit messages.*

8 Huge messages

In all the previous sections, we have run experiments with four message-sizes, namely 1, 10, 30, and 100 flits. (Recall that one flit does not mean one byte, but rather one link. Therefore, a 100-flit message fills up all links between its source and its destination for about 40 steps since the diameter of a 32×32 mesh is 62.) Now, some applications may require multicast of much longer messages as, for instance, a row or a block of a matrix. Such messages can be of size 8 to 16 kbytes, say of roughly a thousand flits. Figure 19 depicts experimental results obtained on messages of size 1000 flits. We have considered two cases determined by the number of destinations that is either small (10 destinations), or large (100 destinations). In both cases, the ratio β is set to 100, and the headers are of variable size. The curves depict the network latency as a function of the injection rate.

For 10 destinations, all algorithms perform more or less the same, but the PMC algorithm whose average network latency is about 33% larger than the latencies of the other algorithms. When the network approaches saturation, the MPMC algorithm seems to show a better behavior. The DP algorithm saturates faster than the other algorithms.

For 100 destinations, the network saturates very fast in all cases. As what could be expected, the DP algorithm performs better than the other algorithms, especially when the network approaches saturation.

Huge message sizes are of course not favorable to sophisticated strategies that are designed to reduce the

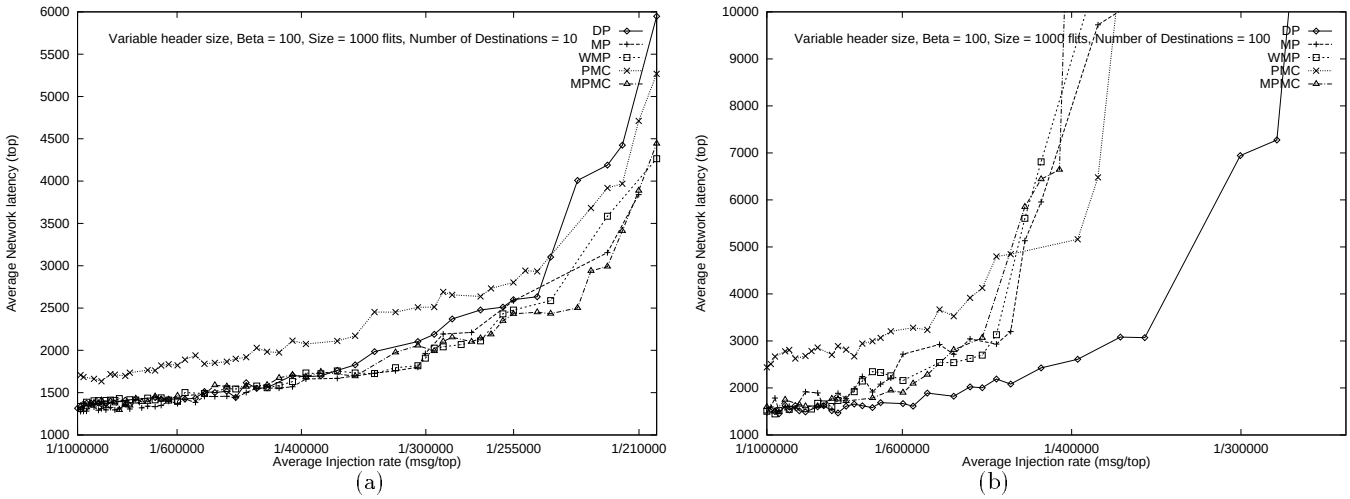


Figure 19: *Network latency as a function of the injection rate for huge messages (1000 flits). On the left hand side, the number of destinations is 10, whereas it is 100 on the right hand side.*

length of the paths up to a small increase of the traffic in the network. Increasing the network load is not dramatic in case of multiple multicast requests. However, it is definitively a penalty when the message size is about 15 times the diameter of the network, and about 25% of the whole bandwidth of the network. In such cases, strategies keeping low traffic are more adapted.

	Off-line computation time
DP	$O(k \log k)$
MP	$O(k \log k)$
SMC	$O(k \log k + \log n(k + \min(k \log k, n \log n)))$
DPMP	$O(k^2)$
WMP	$O(k(\log k + m))$
PMC	$O(k \log k + \log n(k + \min(k \log k, n \log n)))$
MPMC	$O(k \log k + \log n(mk + \min(k \log k, n \log n)))$

Table 1: *Off-line computation times of all the multicast algorithms considered in this paper. Recall that k denotes the number of destination, and that m and n denote the size of the mesh.*

9 Conclusion

Table 1 resumes the off-line computation times of all the algorithms studied in this paper. Figure 20 resumes the average communication times of all the algorithms based on the first set of experiments. When there is no contention, and when the startup time is neglected, the PMC algorithm is, most of the time, the fastest. Concerning off-line computation time, the PMC algorithm is also competitive. For example, if the number of destinations k is small in comparison with the number of columns n ($k = O(n)$), then the off-line computation time of the PMC algorithm is $O(k \log k \log n)$ which leads to a small increase compared with the off-line computation time of the MP algorithm ($O(k \log k)$). Also, if k is larger than n (for instance, when k is proportional to the total number of nodes (mn)) then the off-line complexity of the PMC algorithm is roughly the same as the off-line complexities of both DP and MP algorithms. In the following, we will therefore particularly pay attention to the behavior of the PMC algorithm when contention may occur, when the startup time is not small, or when the message size is huge.

All the experimental results of Section 7 show that the best performances are obtained by the PMC algorithm or either by the DP algorithm or by the MP algorithm. Let us go a bit more in detail. Figure 12 gives a first explanation of the good behavior of the PMC algorithm: the additional traffic is small compared to the other algorithms, excepted when the number of destinations is very big (i.e., about 50% of the nodes). Figures 13, 14 and 15 are dedicated to experiments in which the number of destinations is small. Figure 13 shows that, when the ratio β of the startup time over the propagation time is small, and when the message size is small, the PMC algorithm performs better than the other algorithms when the injection rate increases. In particular, it continues to behave well for injection rates that cause saturation of all the other algorithms (see Figures 13(b)). When the message size increases however, Figure 14 shows that the MP algorithm is better. As shown on Figure 15, these conclusions do not change too much when the startup time increases (100 times the propagation time).

In fact, as shown on Figures 16, 17 and 18 which present experimental results on the average network latency as a function of the number of destinations, the message size (and not the number of destinations) is the real cause of the performance degradation for the PMC algorithm. The PMC algorithm outperforms the other algorithms when the message size is small to medium, and this fact is somewhat independent of the startup time. This was confirmed in Section 8.

When the PMC algorithm does not perform well, that is mainly when the message size is big, the alternative is to use the DP algorithm (see Figures 16, and 17(b)). Actually, if the number of destinations increase, the DP algorithm is often the most efficient, but not always (see for example Figures 17(a) and 18(a)). Now,

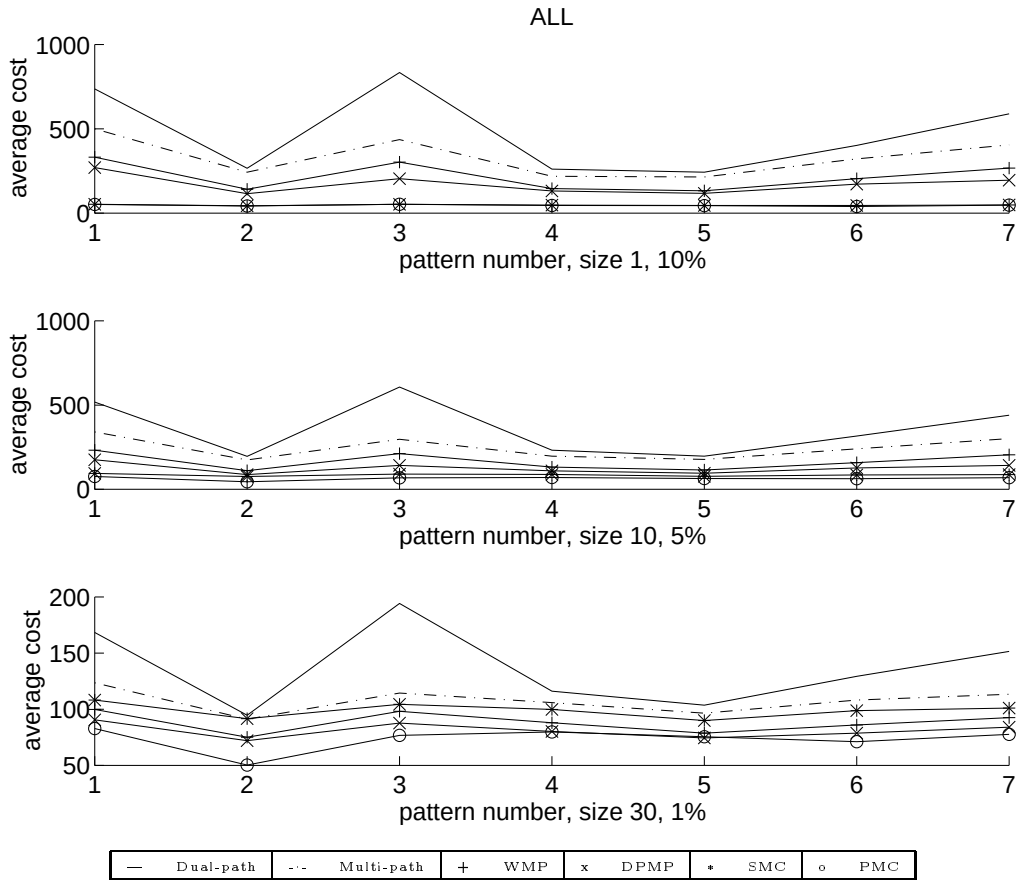


Figure 20: *Experimental comparison between all the algorithms presented in this paper: average number of step as a function of the destination pattern for three samples of message sizes, and three samples of number of destinations. These curves resumes experiments presented on Figures 3 to 11, that is when considering a unique multicast, and when the ratio β is set to 0.*

in the few cases in which the MP algorithm outperforms both DP and PMC algorithms, it is more efficient to use the WMP algorithm. WMP and MP algorithms often show the same behavior in case of contention or when the startup time is important (see Figures 14 and 15), but the WMP algorithm definitively performs better than the MP algorithm when the startup time is small (see Figure 7).

In conclusion, there is not a unique candidate to be the best multicast algorithms on meshes, and the choice depends on many parameters. From our results, we would suggest to consider any of the following algorithms: the DP algorithm of Lin, McKinley, and Ni, the PMC algorithm, and the WMP algorithm. The final choice depends on traffic conditions, and network characteristics.

Acknowledgments: The authors are grateful to Jose Duato for many helpful remarks. They are also thankful to the anonymous referees whose constructive comments greatly improve the quality of the manuscript.

References

[1] K. Aoyama and A. A. Chien. The cost of adaptativity and virtual lanes. *Journal of VLSI Design*, June 1993.

[2] S. Bhattacharya, G. Elsesser, W. T. Tsai, and D. Z. Du. Multicasting in generalized multistage interconnection networks. *Journal of Parallel and Distributed Computing*, 22:80–95, 1994.

[3] C-M. Chiang and L. M. Ni. Multi-address encoding for multicast. In K. Bolding and L. Snyder, editors, *Parallel Computer Routing and Communication, First International Workshop, (PCRCW '94)*, number 853 in Lecture Notes in Computer Science, pages 146–160. Springer-Verlag, May 1994.

[4] A. A. Chien. A cost and speed model for k-ary n-cube wormhole routers. In *Hot Interconnects'93*, Palo Alto, California, August 1993.

[5] W. J. Dally. Virtual-channel flow control. *IEEE Transactions on Parallel and Distributed Systems*, 3(2):194–205, mar 1992.

[6] W. J. Dally and C. L. Seitz. The torus routing chip. *Distributed Computing*, 1(3):87–196, 1986.

[7] W. J. Dally and C. L. Seitz. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Transactions on Computers*, C-36(5):547–553, may 1987.

[8] J. Duato. On the design of deadlock-free adaptative routing algorithms for multicomputers: design methodologies. In *PARLE'91*, Lectures Notes in Computer Science, pages 390–405. Springer-Verlag, June 1991.

[9] J. Duato. Channel classes: a new concept for deadlock avoidance in wormhole networks. *Parallel Processing Letters*, 2(4):347–354, 1992.

[10] J. Duato. A theory of deadlock-free adaptative multicast routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems*, 6(9):976–987, September 1995.

[11] E. Fleury and P. Fraigniaud. Deadlock avoidance in wormhole-routed networks. In *10th International Conference on Parallel and Distributed Computing Systems (PDCS '97)*, New Orleans, Louisiana, USA, 1997.

[12] P. Fraigniaud and C. Gavoille. Memory requirement for universal routing schemes. In 14th *Annual ACM Symposium on Principles of Di stributed Computing (PODC)*, pages 223–230, 1995.

[13] P. Fraigniaud and E. Lazard. Methods and problems of communication in usual networks. *Discrete Applied Mathematics*, 53:79–133, 1994. (special issue on broadcasting).

[14] S. M. Hedetniemi, S. T. Hedetniemi, and A. L. Liestman. A survey of gossiping and broadcasting in communication networks. *Networks*, 18:319–349, 1986.

[15] C-T. Ho and M-Y. Kao. Optimal broadcast on hypercube with wormhole and e-cube routing. In *International Conference on Parallel and Distributed Systems (ICPDS '93)*, pages 694–697, Taipei, Taiwan, December 1993.

[16] D. D. Kandlur and K. G. Shin. Reliable brodcast algorithm for HARTS. *ACM Transactions on Computer Systems*, 9(4):374–398, November 1991.

[17] C. H. Koebel, D. B. Loveman, R. S. Schreiber, G. L. Steele Jr., and M. E. Zosel. *The High Performance Fortran Handbook*. The MIT Press, 1994. ISBN 0-262-11185-3.

[18] Y. Lan, A-H. Esfahanian, and L. Ni. Multicast in hypercube multiprocessor. *Journal of Parallel and Distributed Computing*, 8:30–41, 1990.

[19] X. Lin, A-H. Esfahanian, P. K. McKinley, and A. Burago. Adaptive multicast wormhole routing in hypercube multicomputers. In *Fifth IEEE Symposium on Parallel and Distributed Computing*, Dallas, Texas, December 1993.

[20] X. Lin, P. McKinley, and L. M. Ni. Deadlock-free multicast wormhole routing in 2D-mesh multicomputer networks. In *International Conference on Parallel Processing*, 1991. (see also Technical Report MSU-CPS-ACS-33, 1991).

[21] X. Lin, P. K. McKinley, and A-H. Esfahanian. Adaptive multicast wormhole routing in 2D mesh multicomputers. In *1993 Parallel Architectures and Languages Europe Conference (PARLE '93)*, pages 228–241, Munich, Germany, June 1993.

[22] X. Lin, P. K. McKinley, and L. M. Ni. Deadlock-free multicast wormhole routing in 2D-mesh multicomputers. *IEEE Transactions on Parallel and Distributed Systems*, 5(8):793–804, August 1994.

[23] X. Lin and L. M. Ni. Deadlock-free multicast wormhole routing in multicomputer networks. In *18th Annual International Symposium on Computer Architecture*, pages 116–125, Toronto, May 1991. ACM. (see also Technical Report MSU-CPS-ACS-29, 1990).

[24] X. Lin and L. M. Ni. Multicast communication in multicomputer networks. *IEEE Transactions on Parallel and Distributed Systems*, 4(10):1104–1117, October 1993.

[25] M. D. May, P. W. Thompson, and P. H. Welch, editors. *Networks, Routeurs and Transputers: Function, Performance, and Application*. SGS-THOMSON, 1993.

[26] P. K. McKinley, Y-J. Tsai, and D. F. Robinson. Collective communication in wormhole-routed massively parallel computers. *IEEE Computer*, 28(12):39–50, December 1995.

[27] P. K. McKinley, H. Xu, A-H. Esfahanian, and L. M. Ni. Unicast-based multicast communication in wormhole-routed networks. *IEEE Transactions on Parallel and Distributed Systems*, 5(12):1252–1265, December 1994.

[28] L. M. Ni and P. K. McKinley. A survey of wormhole routing techniques in direct networks. *IEEE Computers*, 26(2):62–76, feb 1993.

[29] D. K. Panda, S. Singal, and R. Kesavan. Multidestination message passing in wormhole k-ary n-cube networks with base routing conformed paths. Technical Report OSU-CISRC-12/9-TR54, Dept. of Computer and Information Science, Ohio State University, 1995. (accepted for publication in IEEE Transactions on Parallel and Distributed Systems).

[30] D. K. Panda, S. Singal, and P. Prabhakaran. Multidestination message passing mechanism conforming to base wormhole routing scheme. In K. Bolding and L. Snyder, editors, *Parallel Computer Routing*

and Communication (PCRCW '94), number 853 in Lecture Notes in Computer Science, pages 131–145. Springer-Verlag, May 1994.

[31] C. B. Stunkel, R. Sivaram, and D. K. Panda. Implementing multidestination worms in switch-based parallel systems: Architectural alternatives and their impact,. In *The 24th Annual IEEE/ACM International Symposium on Computer Architecture (ISCA '97)*, Denver, Colorado, June 1997.

[32] Y. T. Tseng, D. K. Panda, and T Lai. A trip-based multicasting model for wormhole-routed networks with virtual channels. *IEEE Transactions on Parallel and Distributed Systems*, 7(2):138–150, February 1996.

[33] H. Xu, P. K. McKinley, and L. M. Ni. Efficient implementation of barrier synchronization in wormhole-routed hypercube multicomputers. *Journal of Parallel and Distributed Computing*, 16:172–184, 1992.

[34] C. S. Yang, Y. M. Tsai, and C. Y. Liu. Performance evaluation of multicast wormhole routing in 2D-torus multicomputers. In *Fourth International Conference on Computing and Information*, pages 173–178, Toronto Canada, May 1992. IEEE.